

EdgeDecision ed EdgeSTT: controllo vocale locale per Home Assistant con un piccolo modello decisionale non generativo e riconoscimento vocale in streaming

Luca Baratti

4 ottobre 2026

Sommario

I comandi vocali per la casa richiedono decisioni rapide, affidabili e private. I modelli linguistici generativi sono flessibili ma lenti e pesanti per hardware edge come ad esempio un Raspberry Pi e non garantiscono un comportamento prudente; i sistemi a modelli di frase sono veloci ma rigidi. Presentiamo due componenti locali per Home Assistant che lavorano insieme. **EdgeDecision** affronta la comprensione come *scelta* invece che come generazione: per ogni frase assegna un punteggio alle capacità realmente presenti nella casa (per esempio «accendi · TV taverna») e a un candidato esplicito «nessuna azione», e produce una di quattro decisioni: eseguire, chiedere un chiarimento, rifiutare o passare la richiesta a un altro agente. Il suo cuore è **e5s-6L**, un cross-encoder di 23,9 milioni di parametri ricavato da **multilingual-e5-small** (Microsoft, licenza MIT) con riduzione del vocabolario, potatura degli strati da 12 a 6, quattro teste, distillazione da un insegnante di 568 milioni di parametri e quantizzazione INT8 (24,3 MB), addestrato solo su dati sintetici in nove lingue. **EdgeSTT** porta sul Raspberry Pi 5 il riconoscimento vocale in streaming di NVIDIA Nemotron 3.5 ASR (0,6 miliardi di parametri, INT8) tramite sherpa-onnx e il protocollo Wyoming: l'audio viene trascritto mentre l'utente parla e a fine frase resta da elaborare solo l'ultimo blocco. Con il modello risultante dal settimo ciclo (7c) e il software della versione 1.0, EdgeDecision decide correttamente 805 frasi scritte a mano su 846 (95%) in nove lingue, senza alcuna esecuzione sbagliata e confermando sempre le azioni ad alto rischio; sulle stesse frasi l'agente predefinito di Home Assistant ne decide correttamente 370 (44%) ed esegue 15 azioni ad alto rischio senza chiedere conferma. Su un Raspberry Pi 5 con un satellite Voice PE la catena dalla fine della frase alla decisione richiede in mediana circa 0,48 s (276 ms per completare la trascrizione e 203 ms per decidere), mentre la sola trascrizione con Faster-Whisper small richiedeva circa 5 s.

1. Da dove nasce il progetto

L'idea: un modello «System 1». Il 15 settembre 2026 TypeSafe AI ha presentato Jev, un modello che non conversa e non scrive testo. Riceve uno stato e una domanda e restituisce una scelta, un voto o una probabilità, con la relativa confidenza. Il suo fondatore lo ha descritto come un modello «System 1», riprendendo la distinzione di Daniel Kahneman tra due modi di pensare. Il System 1 è veloce, automatico e intuitivo: riconoscere un volto, capire che la pasta è cotta. Il System 2 è lento, deliberato e faticoso: fare un calcolo, pianificare un viaggio. Un modello linguistico generativo affronta ogni richiesta come un System 2, anche la più banale, perché ragiona scrivendo un token dopo l'altro. Un modello System 1 si limita a pesare alternative già note, e può pesarle tutte insieme. Pochi giorni dopo Jev è comparso CLM-8B, un modello aperto costruito sullo stesso principio.

La domanda. Jev è un servizio cloud e CLM-8B richiede una GPU. Da qui nasce la domanda: un modello System 1 specializzato in un solo dominio può diventare abbastanza piccolo da girare in casa, su un Raspberry Pi, senza perdere affidabilità? La domotica è un caso adatto, perché quasi tutto ciò

che si chiede a un assistente vocale domestico è una decisione da System 1. «Accendi la luce della cucina» non richiede ragionamento: richiede di scegliere la cosa giusta tra le poche centinaia che la casa sa fare.

Il problema da risolvere. Chi usa Home Assistant con la voce oggi deve scegliere tra due compromessi:

- **Riconoscimento a modelli di frase** (incluso in Home Assistant): è veloce e locale, ma capisce solo le formulazioni previste. Basta un «porta al massimo la luce del salotto» o un errore del riconoscimento vocale per non capire più.
- **Agente LLM**: capisce frasi libere. Se è in locale, però, su un Raspberry Pi impiega secondi e occupa gigabyte. Se è nel cloud, dipende dalla rete, ha un costo per richiesta e porta fuori casa ciò che si dice in casa. In entrambi i casi, generando testo, può inventare un dispositivo o un'azione che non esiste.

Servono quindi un sistema **veloce** (una risposta in meno di un quarto di secondo), **locale** (funziona senza internet e non manda fuori le frasi), **sicuro** (non esegue mai l'azione sbagliata su una serratura o un cancello, e quando non è certo chiede) e **multilingue**. L'ultimo punto conta per le case con ospiti di lingue diverse: seconde case, appartamenti in affitto, hotel.

La divisione del lavoro. EdgeDecision fa da System 1 della casa. Decide da solo i comandi diretti e le domande sullo stato, che sono la grande maggioranza. Quando una richiesta è da System 2 (una condizione come «se domani piove chiudi le tende», una domanda aperta, una conversazione) la riconosce come tale e la passa a un LLM o all'agente di Home Assistant. Non deve saper fare tutto: deve essere veloce e sicuro su ciò che gli compete, e onesto su ciò che non gli compete. Alcuni esempi:

- «accendi la luce della cucina», «la finestra del bagno è aperta?», «metti radio deeJay in salotto»: System 1, EdgeDecision esegue o risponde in circa due decimi di secondo sul Raspberry Pi;
- «accendi la tv» con due televisori e il microfono in una stanza senza tv: chiede quale (CLARIFY);
- «apri la porta d'ingresso»: chiede conferma, perché l'azione è a rischio;
- «se domani piove chiudi le tende», «consigliami una ricetta»: System 2, passa la frase all'LLM (ESCALATE).

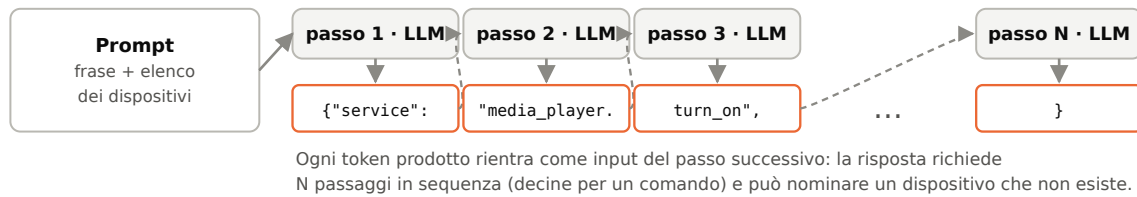
La voce. Le prime prove sul Raspberry Pi hanno mostrato che, una volta resa veloce la decisione, il tempo si perdeva altrove: nel riconoscimento vocale. Faster-Whisper small, il motore locale più usato con Home Assistant, impiegava circa 5 secondi per trascrivere un comando breve, perché comincia a lavorare solo quando l'utente ha finito di parlare. Accanto a EdgeDecision è nato quindi EdgeSTT, un add-on che usa un modello di riconoscimento in streaming e trascrive *mentre* l'utente parla. Questo documento descrive entrambi: la versione 1.0 li pubblica insieme, nello stesso repository, con l'integrazione che rende EdgeDecision l'agente di conversazione di Assist.

2. Introduzione

Un assistente vocale domestico deve trasformare una frase come «abbassa un po' la tapparella della cucina» in una chiamata precisa a un servizio della casa. Le soluzioni disponibili si dividono in due famiglie. I **sistemi a modelli di frase** (come il riconoscimento degli intenti incluso in Home Assistant) sono veloci e prevedibili, ma riconoscono solo le formulazioni previste e tollerano male gli errori del riconoscimento vocale. I **modelli linguistici generativi** (LLM) capiscono formulazioni libere, ma su un Raspberry Pi impiegano secondi per risposta, occupano gigabyte di memoria e, generando testo, possono inventare un'azione o un dispositivo che non esiste.

EdgeDecision segue una terza strada: tratta il problema come una **scelta tra alternative note**. Il catalogo della casa viene trasformato in un elenco di *capacità* (un'azione su un bersaglio, per esempio «imposta posizione · Tapparella cucina») e un piccolo modello neurale assegna un punteggio a ciascuna. Il modello non genera nulla: può solo preferire una capacità esistente o il candidato speciale «nessuna azione». Sopra il punteggio, una politica calibrata decide se eseguire, chiedere, rifiutare o delegare, privilegiando la prudenza: chiedere conferma costa un secondo, eseguire l'azione sbagliata può costare molto di più. La Figura 1 confronta i due approcci su un esempio reale.

A · LLM generativo (autoregressivo)



B · EdgeDecision (non autoregressivo)

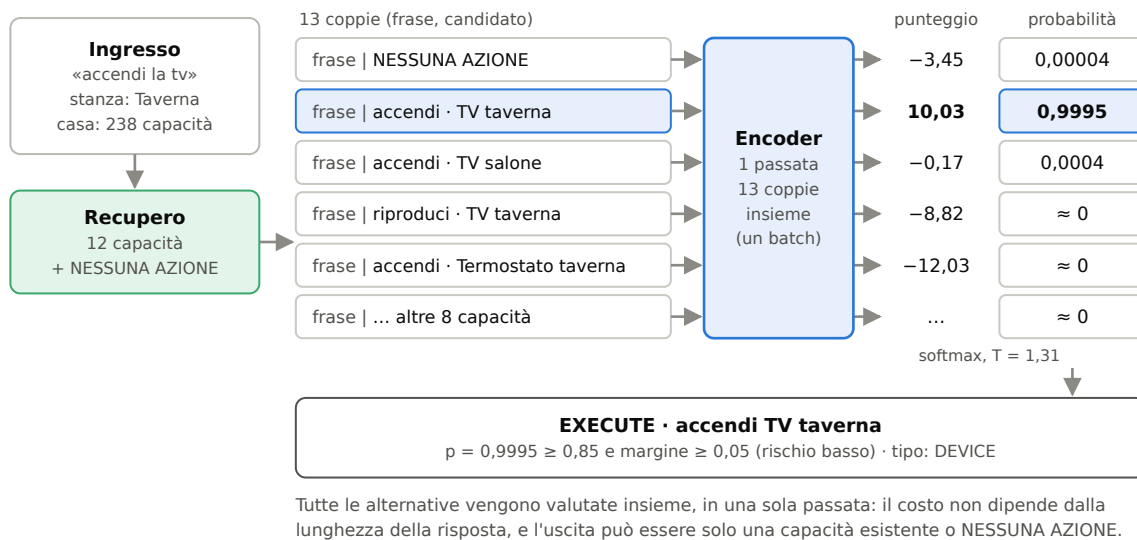


Figura 1. Generazione autoregressiva e scelta in parallelo. (A) Un LLM scrive la risposta un token alla volta e ogni passo dipende dal precedente (i token sono un esempio). (B) EdgeDecision forma una coppia (frase, candidato) per ciascuna delle 12 capacità recuperate più NESSUNA AZIONE e le valuta tutte insieme in un'unica passata dell'encoder; una softmax trasforma i punteggi in probabilità e la politica decide. Punteggi e probabilità sono quelli reali del modello a 6 strati per «accendi la tv» detta in taverna nella casa di prova. Il recupero dei candidati richiede un'altra passata dell'encoder, sulla sola frase.

I contributi di questo lavoro sono:

- una formulazione del controllo vocale come ordinamento delle capacità presenti *a runtime*, con un candidato «nessuna azione» esplicito e quattro esiti (EXECUTE, CLARIFY, REJECT, ESCALATE);
- una rete unica con quattro teste (punteggio, tipo di richiesta, embedding per il recupero, etichette delle parole), esportata in un solo grafo ONNX che gira senza PyTorch;
- una procedura per arrivare da un insegnante di 568 M parametri a uno studente di 23,9 M parametri (e5s-6L, 6 strati; fino al ciclo 5 anche una variante a 4 strati) tramite dati sintetici, parafrasi verificate, distillazione, potatura del vocabolario e degli strati, quantizzazione INT8 e calibrazione;

- EdgeSTT, un add-on di riconoscimento vocale in streaming per il Raspberry Pi 5, con silenzio aggiunto in testa e un flush deterministico in coda, che porta la catena dalla fine della frase alla decisione a circa mezzo secondo;
- una valutazione su insiemi scritti a mano in nove lingue, su dati sintetici e sul campo, con un Raspberry Pi 5, un satellite Voice PE e Home Assistant, e un confronto con l'agente predefinito di Home Assistant sulle stesse 846 frasi e con lo stesso criterio di correttezza.

3. Lavori correlati

Ri-ordinamento con cross-encoder. Valutare insieme domanda e documento con un Transformer, invece di confrontare due vettori calcolati separatamente, è la tecnica standard per ri-ordinare i risultati di una ricerca. EdgeDecision la applica a «documenti» molto corti: le descrizioni testuali delle capacità della casa. Poiché lo spazio delle uscite è testo, aggiungere un dispositivo non richiede di riaddestrare il modello.

Modelli multilingue compatti. Lo studente parte da `multilingual-e5-small`, un encoder di 12 strati e 384 dimensioni addestrato per embedding di frasi in circa 100 lingue, che usa il vocabolario SentencePiece di XLM-RoBERTa. L'insegnante è `bge-reranker-v2-m3`, un cross-encoder multilingue della famiglia BGE-M3.

Compressione. La distillazione della conoscenza addestra un modello piccolo a imitare le distribuzioni di un modello grande. Rimuovere strati da un Transformer pre-addestrato e poi riaddestrarlo conserva gran parte dell'accuratezza. La calibrazione con temperatura rende le probabilità interpretabili, requisito per le soglie di sicurezza.

Modelli decisionali «System 1». Come descritto all'inizio, nel settembre 2026 sono stati presentati modelli generali che assegnano punteggi a scelte invece di generare testo, come Jev (servizio cloud commerciale) e CLM-8B (8 miliardi di parametri, aperto, per GPU). EdgeDecision condivide il principio ma è specializzato in un solo dominio, circa 400 volte più piccolo di CLM-8B, e gira in locale su un Raspberry Pi. Non abbiamo confrontato sperimentalmente i tre sistemi.

Riconoscimento vocale in streaming. Whisper è un modello codificatore-decodificatore che trascrive un segmento audio completo: in un assistente vocale il calcolo comincia solo a frase finita. I modelli con trasduttore RNN-T emettono invece il testo man mano che arriva l'audio. FastConformer riduce il costo dell'encoder Conformer e la sua variante *cache-aware* lo rende adatto allo streaming, conservando tra un blocco e l'altro lo stato degli strati invece di ricalcolare il contesto. Nemotron 3.5 ASR Streaming 0.6B è un modello multilingue di questo tipo, che EdgeSTT esegue con il runtime sherpa-onnx.

Comprensione a modelli di frase. L'agente di conversazione predefinito di Home Assistant riconosce gli intenti con la libreria `hassil`, confrontando la frase con i modelli di frase del progetto `home-assistant-intents`, scritti a mano dalla comunità per ogni lingua. È il riferimento con cui confrontiamo EdgeDecision nella sezione «Confronto con l'agente predefinito di Home Assistant».

4. Formulazione del problema

L'ingresso è una frase u (il testo prodotto dal riconoscimento vocale), l'area a del microfono, se nota, e il catalogo C della casa: aree, entità con nome, alias, classe e stato, e le azioni che ciascuna supporta. Dal catalogo si ricava l'insieme delle capacità $K = \{k_1, \dots, k_n\}$; ogni capacità è una coppia `azione@bersaglio`, dove il bersaglio è un'entità (`light.lampada_arco`) oppure un gruppo («tutte le luci del salone»). La casa di prova usata nei test ha 51 entità e circa 238 capacità.

Ogni capacità è resa come testo, in inglese e con i nomi originali dei dispositivi, per esempio:

```
media player turn on: turn on a TV / speaker / media player || target: TV taverna (media player tv) || area: Taverna || state: off
```

A queste si aggiunge il candidato NO_ACTION («none of the available actions matches this request»), così il modello non è mai costretto a scegliere un dispositivo. L'uscita è una decisione $d \in \{\text{EXECUTE}, \text{CLARIFY}, \text{REJECT}, \text{ESCALATE}\}$, con la capacità scelta, i parametri (percentuali, gradi, colori...) e una confidenza calibrata. La Tabella 1 mostra esempi reali di decisioni sulla casa di prova.

Tabella 1. Esempi di decisioni nella casa di prova (scenari verificati sul Raspberry Pi).

Frase (stanza del microfono)	Decisione	Effetto
accendi le luci del salone	EXECUTE	accende le due luci del salone
spegni la tv (in Taverna)	EXECUTE	spegne la TV della taverna, non quella del salone
accendi la tv (stanza ignota, due TV)	CLARIFY	«Intendi TV salone o TV taverna?»
apri il portoncino	CLARIFY	chiede conferma: azione ad alto rischio
imposta il termostato della zona giorno	CLARIFY	chiede il valore mancante
non accendere la luce del box	REJECT	non fa nulla (negazione)
chiudi il velux e accendi il climatizzatore	EXECUTE	due comandi eseguiti
se piove chiudi la tenda del terrazzo	ESCALATE	regola condizionale: passa a un LLM

5. Architettura del sistema

La Figura 2 mostra il percorso di una frase. La rete neurale calcola i punteggi nel passo 4 e gli embedding usati dal recupero nel passo 3; gli altri passi sono deterministici e servono a proteggere, spiegare e completare la decisione del modello.

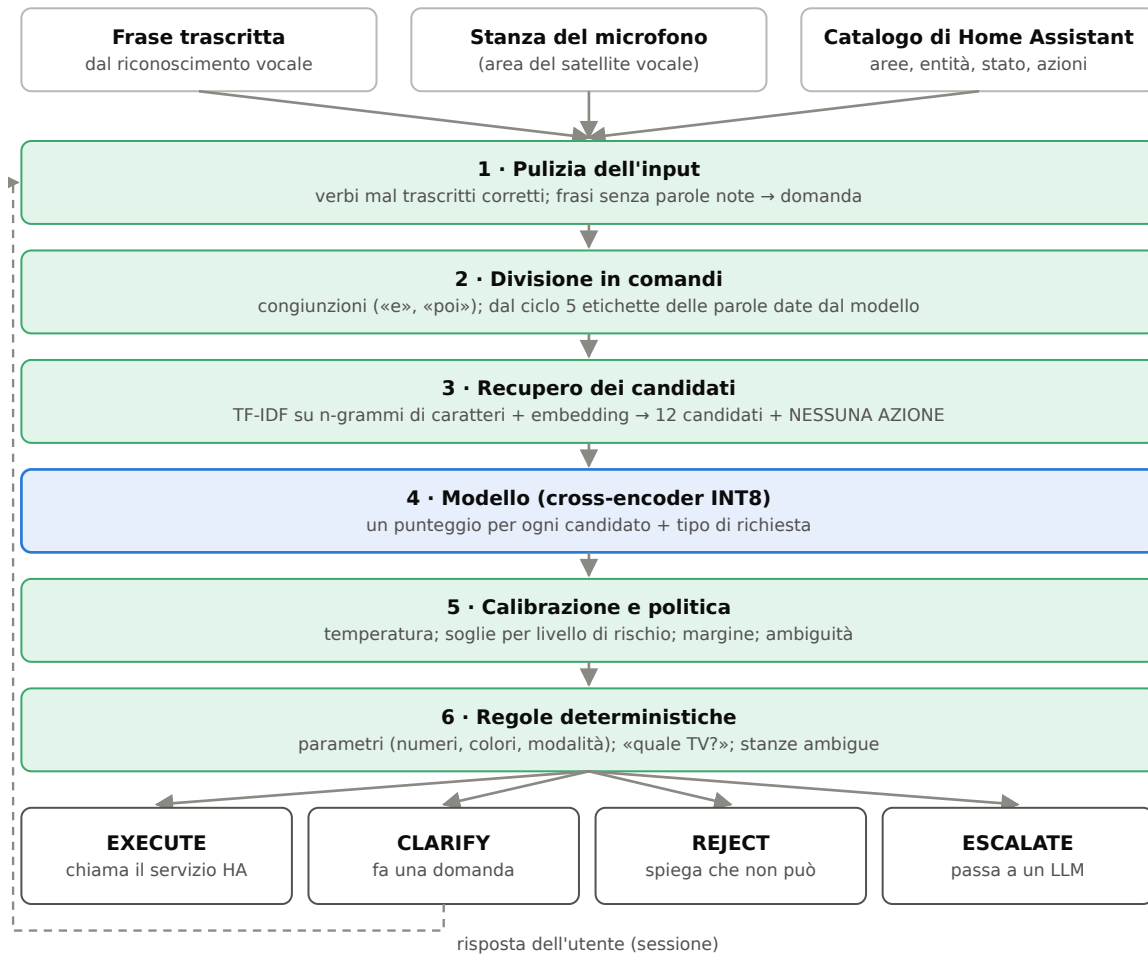


Figura 2. Percorso di una frase a runtime. In blu il modello neurale; in verde i passi deterministici del software EdgeDecision. Una domanda (CLARIFY) apre una sessione legata al satellite vocale: la risposta dell'utente («quella della taverna», «sì», «al 40%») viene interpretata nel contesto della domanda.

5.1 Recupero dei candidati

Per non valutare centinaia di capacità a ogni frase, un recupero ibrido ne seleziona 12. Combina un indice TF-IDF su n-grammi di 3 e 4 caratteri, robusto agli errori di battitura e di trascrizione, con la similarità coseno tra l'embedding della frase e quelli delle capacità. Gli embedding delle capacità non contengono lo stato e vengono ricalcolati solo quando cambia il catalogo, quindi per ogni frase serve una sola passata dell'encoder. Il recupero contiene la capacità corretta nel 99,9% delle frasi di validazione e nel 100% delle frasi gold. La Sezione 8 mostra il procedimento completo su un esempio, con i numeri reali.

5.2 Punteggio e tipo di richiesta

Il modello valuta le coppie (frase, capacità) e produce un punteggio per ciascun candidato; la coppia (frase, NO_ACTION) alimenta anche una seconda testa che classifica la richiesta come comando per un dispositivo (DEVICE), fuori tema (OUT_OF_DOMAIN), complessa (COMPLEX, per esempio una condizione o un'azione pianificata) o senza azione (NOOP, per esempio una negazione). Il quinto ciclo aggiunge CONFIRM (una risposta come «sì») e HISTORY («cosa hai fatto?»).

5.3 Calibrazione e politica di decisione

I punteggi dei candidati passano per una softmax con temperatura T , stimata sul validation con il modello INT8 e il recupero reale (per la variante a 6 strati $T = 1,31$ nel ciclo 4 e 1,52 nel ciclo 5). La politica applica poi, nell'ordine: il tipo di richiesta; l'ambiguità (se un candidato *incompatibile* con il

primo supera il 30% della sua probabilità, si chiede quale dei due); le soglie di probabilità e di margine, che dipendono dal rischio dell'azione (Tabella 2); la disponibilità dell'entità e la validità dei parametri. Le azioni ad alto rischio (sbloccare una serratura, disattivare l'allarme, aprire il garage) chiedono sempre conferma.

Tabella 2. Soglie di esecuzione scelte dalla calibrazione. Si esegue solo se la probabilità calibrata e il margine sulle alternative incompatibili superano entrambe le soglie.

Rischio dell'azione	Esempi	Probabilità minima	Margine minimo
nessuno	leggere uno stato	0,70	0,00
basso	luci, tapparelle, clima, TV, scene	0,85	0,05
medio	chiudere a chiave, inserire l'allarme	0,95	0,17
alto	aprire serrature, garage e cancelli; disinserire l'allarme	0,97 + conferma	0,25

5.4 Regole deterministiche

Il modello decide *cosa* fare; i valori vengono estratti con regole: numeri anche scritti in lettere, percentuali, gradi, variazioni relative («un po' più su» rispetto allo stato attuale), colori e modalità. Alcune regole controllano *quale* dispositivo è indicato: se la frase contiene solo un nome generico («accendi la tv») e in casa ci sono più dispositivi di quel tipo, il sistema chiede invece di affidarsi alla preferenza appresa dal modello; se una parola della frase appartiene a un solo candidato («i faretto dell'isola») quel candidato vince; se una parola indica più stanze («bagno» con due bagni) si chiede quale. Una frase senza alcuna parola nota (un tipico errore del riconoscimento vocale) non viene mai eseguita. Dalla versione 0.8 altre regole coprono i casi emersi nell'uso reale: una frase che dice di sbloccare non può chiudere a chiave, un elettrodomestico nominato ma assente (la lavastoviglie) non ne accende un altro (la lavatrice), un oggetto nominato che non compare nel nome del dispositivo scelto («lucernario» per «Luce mansarda») porta a una domanda, e «che ore sono?» o «che giorno è?» ricevono la risposta dall'orologio del Raspberry.

5.5 Integrazione con Home Assistant

EdgeDecision gira come add-on di Home Assistant (un contenitore con `onnxruntime`, `tokenizers` e `numpy`, senza `PyTorch`). All'avvio legge aree, entità e alias tramite le API REST e WebSocket, e ogni 60 secondi controlla se il catalogo è cambiato. Un componente personalizzato lo registra come *agente di conversazione* di Assist: le decisioni EXECUTE chiamano i servizi, CLARIFY mantiene aperto il microfono per la risposta, ESCALATE passa la frase a un secondo agente (per esempio un LLM), e ogni decisione viene pubblicata come evento per il debug. Il testo arriva da EdgeSTT, descritto nella prossima sezione, o da qualunque altro motore di riconoscimento vocale configurato in Assist; l'integrazione passa all'add-on anche il dispositivo del satellite che ha sentito la frase, così le risposte e la musica vanno all'altoparlante giusto.

6. EdgeSTT: il riconoscimento vocale in streaming

EdgeSTT è il secondo add-on del progetto: trasforma in testo l'audio del satellite vocale. Usa NVIDIA Nemotron 3.5 ASR Streaming 0.6B, un modello FastConformer *cache-aware* con decodifica RNN-T e 600 milioni di parametri, nell'esportazione ONNX INT8 preparata dal progetto sherpa-onnx, e parla con Assist tramite il protocollo Wyoming. Riconosce le stesse nove lingue di EdgeDecision; la lingua arriva da Assist con ogni richiesta ed entra nel modello come indicatore di lingua.

6.1 Perché lo streaming

Il tempo che conta per chi parla è quello tra la fine della frase e la risposta. Un modello come Whisper trascrive solo a frase finita, quindi tutto il suo calcolo cade in quell'attesa: sul Raspberry Pi 5, nelle prove fatte prima di questo progetto, Faster-Whisper small (INT8) impiegava circa 4,8–5,0 s per un comando breve e sbagliava anche parole semplici («spegni la TV» → «spenia la tv»). Nemotron elabora invece l'audio a blocchi fissi mentre l'utente parla (Figura 3). Con blocchi da 560 ms il fattore di tempo reale (RTF) sul Pi 5 è circa 0,34–0,36: ogni secondo di voce richiede circa un terzo di secondo di calcolo, svolto durante il parlato. Quando l'utente smette resta solo la *finalizzazione*, cioè il calcolo degli ultimi uno o due blocchi.

Blocchi più piccoli costano troppo sul Pi (RTF circa 0,9–1,0 a 160 ms e 1,8 a 80 ms); blocchi da 1120 ms dimezzano l'RTF (circa 0,18–0,22) ma allungano l'attesa dell'ultimo blocco. Il valore predefinito è 560 ms. Un modello CTC più leggero (Omnilingual ASR 300M, RTF circa 0,23 sul Pi) è stato scartato perché sbagliava troppe parole («disattiva la TV» → «disativala tivur»). Il profilo del calcolo mostra inoltre che l'encoder occupa circa il 90% del tempo: sostituire il decodificatore RNN-T con una testa CTC farebbe risparmiare solo il 4–6%, mentre la leva vera è la dimensione del blocco.

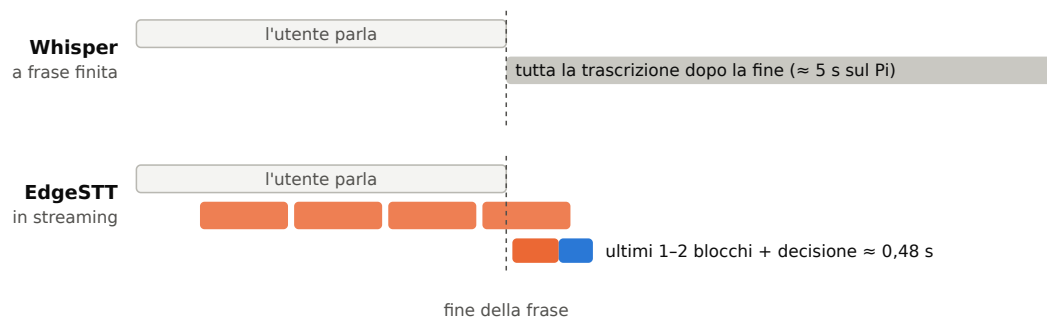


Figura 3. Dove cade il calcolo del riconoscimento vocale (schema; scala dei tempi indicativa). In alto, un modello che trascrive a frase finita lavora tutto dopo la fine della frase. In basso, EdgeSTT elabora blocchi da 560 ms (arancione) mentre l'utente parla; a frase finita restano gli ultimi blocchi (finalizzazione, 276 ms in mediana sul Pi 5) e la decisione di EdgeDecision (blu, 203 ms in mediana).

6.2 Inizio e fine della frase

Due correzioni nel server evitano di perdere la prima e l'ultima parola. **In testa**, all'inizio di ogni richiesta EdgeSTT aggiunge 300 ms di silenzio: senza, una parola detta subito dopo l'attivazione veniva tagliata («Alles hat ein Ende...» → «hat ein Ende...») e una parola isolata dava un testo vuoto. **In coda**, il runtime scarta l'ultimo blocco incompleto; alla fine dell'audio il server aggiunge quindi silenzio finché i blocchi elaborati coprono tutto l'audio più 0,3 s, più circa 0,1 s di contesto destro, e solo allora chiude il flusso (*flush deterministico*). In una prova con 3 registrazioni e 6 posizioni della fine del parlato rispetto ai blocchi, senza flush l'ultima parola si perdeva in 2 casi su 18 con blocchi da 560 ms e in 8 su 18 con blocchi da 1120 ms; con il flush in nessuno.

6.3 Accuratezza e limiti del modello

Per blocchi da 560 ms NVIDIA dichiara un tasso di errore sulle parole (WER) sul test FLEURS del 4,4% in italiano, 4,3% in spagnolo, 5,7% in portoghese, 8,0% in inglese, 8,4% in tedesco, 9,5% in francese e 12,0% in olandese; polacco e svedese sono nel livello a copertura più ampia, con un'accuratezza molto inferiore. Non abbiamo ancora misurato il WER su registrazioni reali di comandi domestici. Su 150 frasi di domotica in italiano lette da una voce sintetica non italiana il WER è circa il 31%: un valore che misura insieme il modello e la qualità dell'audio sintetico, e che non va letto come accuratezza su parlanti reali. Lo stesso test mostra però i comportamenti che contano per

EdgeDecision: le risposte di una o due parole («caldaia», «taverna») escono vuote nel 16,7% dei casi, i numeri possono uscire in cifre o in lettere («diciannove minuti» → «19 minuti») e il testo contiene maiuscole e punteggiatura. EdgeDecision accetta tutte queste forme e tratta un testo vuoto come «non ho capito», senza eseguire nulla.

Tabella 3. EdgeSTT in breve. Le misure di tempo e memoria sono sul Raspberry Pi 5.

Voce	Valore
Modello	NVIDIA Nemotron 3.5 ASR Streaming 0.6B: FastConformer cache-aware con RNN-T, 600 M parametri
Formato e runtime	ONNX INT8 (sherpa-onnx 1.13.8), decodifica greedy su CPU
Download al primo avvio / spazio su disco	circa 475 MB / 682 MB
Blocchi	560 ms (predefinito) o 1120 ms
RTF	circa 0,34–0,36 (560 ms); 0,18–0,22 (1120 ms)
Finalizzazione, 50 comandi reali (560 ms, 4 thread)	mediana 276 ms; 90% sotto 519 ms
Memoria	1,40 GB
Lingue	it, en, es, fr, de, nl, pt, pl, sv, più il rilevamento automatico
Collegamento con Home Assistant	protocollo Wyoming, porta 10300

EdgeSTT usa il modello base di NVIDIA senza modifiche: un addestramento sui comandi domestici e sulle risposte brevi è pianificato ma non fa parte della versione 1.0. Il modello non è incluso nel repository: l'add-on lo scarica al primo avvio dalla release pubblica di sherpa-onnx.

7. Dal comando vocale all'azione: chi fa cosa

EdgeDecision non è solo il modello neurale. Il modello fa una cosa sola, dare punteggi; tutto il resto è software deterministico scritto in Python, affiancato dai componenti di Home Assistant che gestiscono microfono, voce e dispositivi (Figura 4). In sintesi:

- **Home Assistant** riceve l'audio dal satellite vocale (Voice PE), lo fa trascrivere (STT), passa il testo all'agente di conversazione scelto, pronuncia la risposta (TTS) e, su richiesta, accende o spegne i dispositivi.
- **EdgeSTT**, il secondo add-on, è il riconoscimento vocale: riceve l'audio in streaming tramite Wyoming e restituisce il testo ad Assist.
- **L'integrazione EdgeDecision**, un piccolo componente installato in Home Assistant, è l'agente di conversazione: trova la stanza del microfono, invia la frase all'add-on, riceve la risposta e la restituisce ad Assist; le richieste ESCALATE le passa a un altro agente (per esempio un LLM).
- **Il software dell'add-on** legge il catalogo della casa, tiene aggiornati gli stati, pulisce e divide le frasi, sceglie i candidati, applica la politica e le regole, chiama i servizi di Home Assistant e scrive la frase di risposta.
- **Il modello** trasforma una frase in un vettore (per il recupero) e dà un punteggio alle coppie (frase, candidato), più il tipo di richiesta. Non chiama servizi, non conosce le soglie e non parla con Home Assistant.

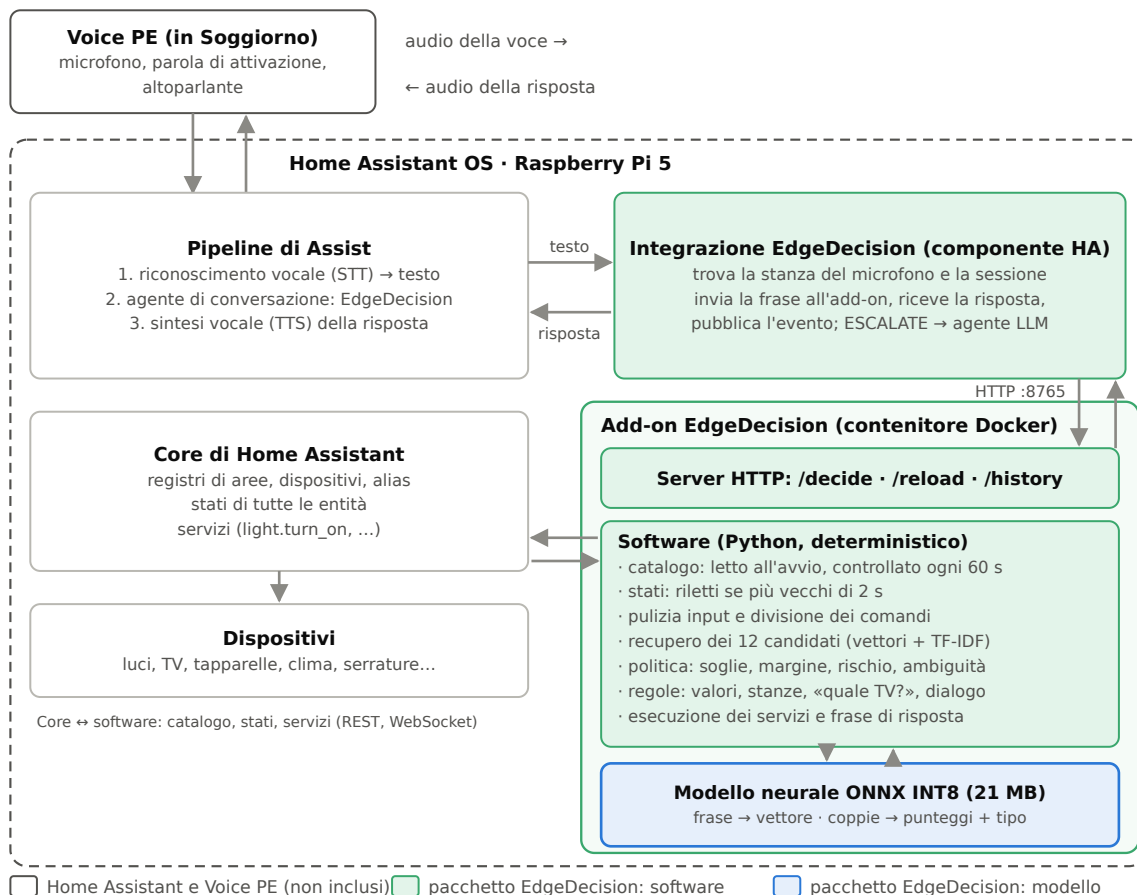


Figura 4. Componenti del sistema su un Raspberry Pi 5 con Home Assistant OS. L'add-on è un contenitore Docker con un server HTTP sulla porta 8765; il modello ONNX gira dentro l'add-on e viene usato solo dal software. In verde il software del pacchetto EdgeDecision (integrazione e add-on), in blu il modello; in bianco i componenti di Home Assistant, che non fanno parte del pacchetto.

7.1 Primo esempio: «accendi luce cucina»

La Figura 5 mostra i 18 passaggi reali, dal momento in cui l'utente parla al Voice PE del soggiorno fino alla risposta vocale. L'integrazione invia all'add-on una richiesta come questa:

```
POST /decide {"text": "accendi luce cucina", "context_area": "soggiorno", "session": "<id del satellite>", "execute": true, "lang": "it"}
```

Il software rilegge gli stati se sono più vecchi di 2 secondi, verifica che la frase contenga parole note e nota che «cucina» è il nome di una stanza: la stanza del microfono (Soggiorno) viene quindi ignorata. Seguono le due passate del modello descritte nella Sezione 8. La capacità vincente è «accendi tutte le luci della Cucina» con probabilità 0,9975; il rischio è basso (soglia 0,85) e la politica decide EXECUTE. Il software traduce la capacità in una chiamata di servizio:

```
POST /api/services/light/turn_on {"entity_id": ["light.pendente_tavolo", "light.faretti_isola", "light.striscia_mensola"]}
```

Home Assistant accende le tre luci; l'add-on restituisce all'integrazione la decisione e la risposta «Fatto.», l'integrazione pubblica l'evento `edgedecision_decision` e Assist pronuncia la risposta. Il modello interviene solo nei passaggi 7–8 e 10–11.

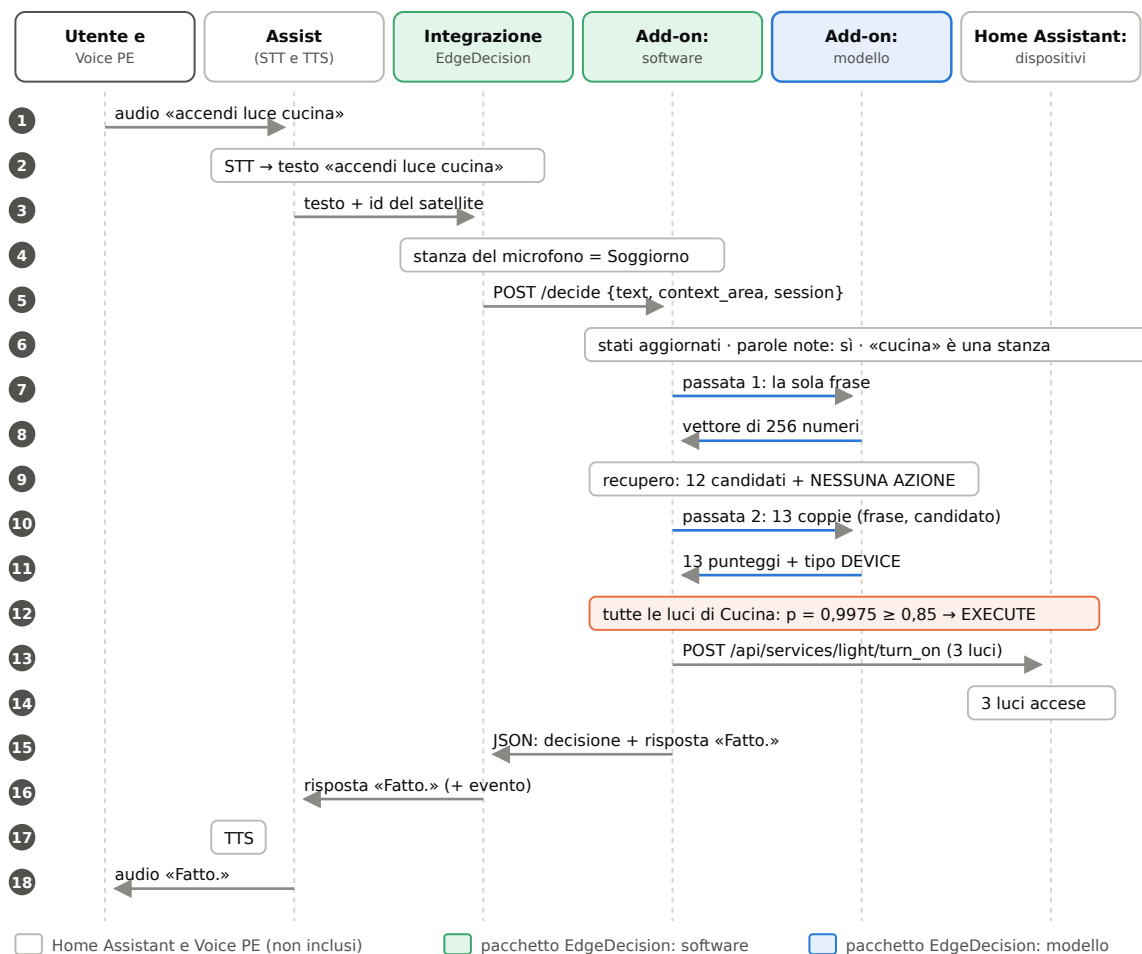


Figura 5. Flusso di «accendi luce cucina» pronunciato in soggiorno. Colonne verdi: software del pacchetto EdgeDecision; colonna blu: il modello; colonne bianche: Voice PE e Home Assistant. Freccie blu: le due passate del modello; riquadro arancione: la decisione della politica. Capacità, probabilità, entità e servizi sono quelli prodotti dal sistema sulla casa di prova (modello a 6 strati, ciclo 4).

La Figura 6 scende di un livello: divide l'add-on nei suoi moduli (server HTTP, motore, modello, adattatore verso Home Assistant) e mostra ogni messaggio con la sua direzione e il suo canale. La Tabella 4 riporta, per ciascun messaggio, il contenuto esatto.

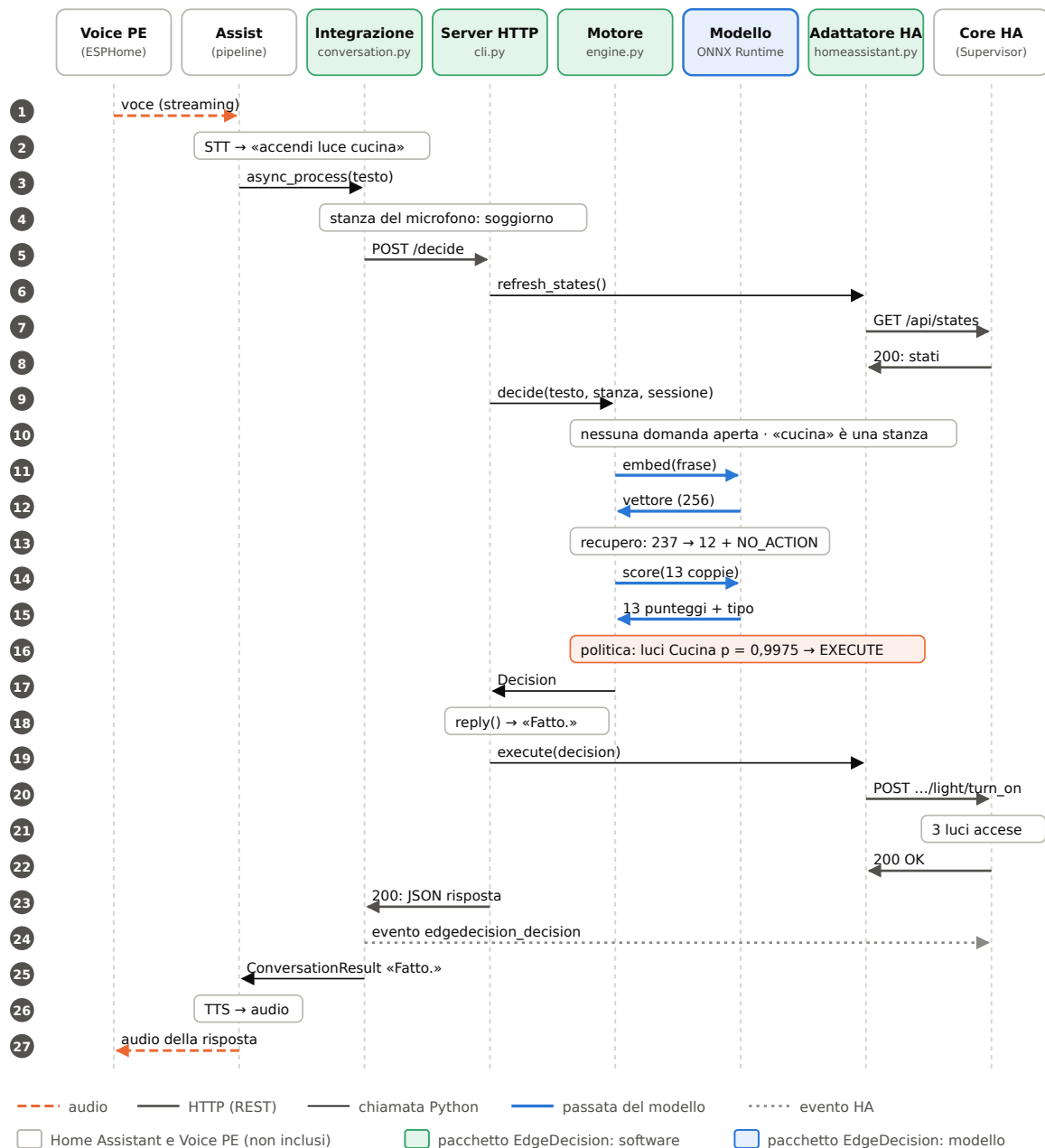


Figura 6. «accendi luce cucina», messaggio per messaggio. Ogni colonna è un componente (verde: pacchetto EdgeDecision; blu: modello; bianco: Voice PE e Home Assistant); ogni freccia è un messaggio, dal mittente al destinatario. Tratteggio arancione: audio; grigio scuro: HTTP; nero sottile: chiamate tra funzioni Python dentro l'addon; blu: passate del modello; puntinato: evento di Home Assistant. I nomi delle funzioni sono quelli reali del codice.

Tabella 4. I 27 messaggi della Figura 6: chi li manda, a chi, con quale canale e con quale contenuto (valori reali della casa di prova).

#	Da → a	Canale	Contenuto
1	Voice PE → Assist	audio (ESPHome)	la voce, dopo la parola di attivazione
2	Assist	STT (stt.edgestt_nemotron)	testo «accendi luce cucina»
3	Assist → Integrazione	Python: <code>async_process()</code>	testo, lingua «it», id del satellite
4	Integrazione	registri di HA	area del satellite → «soggiorno»
5	Integrazione → Server	HTTP POST <code>http://local-edgedecision:8765/decide</code>	<code>{ "text": "accendi luce cucina", "context_area": "soggiorno", "session": "<id satellite>", "execute": true, "lang": "it" }</code>

#	Da → a	Canale	Contenuto
6	Server → Adattatore	Python: <code>refresh_states()</code>	solo se gli stati hanno più di 2 s
7	Adattatore → Core	HTTP GET <code>http://supervisor/core/api/states</code>	con il token del Supervisor
8	Core → Adattatore	HTTP 200, JSON	stato di ogni entità (per esempio <code>light.pendente_tavolo: off</code>)
9	Server → Motore	Python: <code>engine.decide()</code>	testo, «soggiorno», sessione
10	Motore	—	nessuna domanda in sospeso per la sessione; parole tutte note; «cucina» è un'area → la stanza del microfono non si usa
11	Motore → Modello	ONNX Runtime <code>session.run</code> , 1 testo	«accendi luce cucina»
12	Modello → Motore	uscita <code>embedding</code>	256 numeri
13	Motore	numpy	$0,7 \times \text{coseno}$ con i 237 vettori delle capacità + $0,3 \times \text{TF-IDF}$ → 12 capacità + <code>NO_ACTION</code>
14	Motore → Modello	<code>session.run</code> , 13 coppie in un lotto	<code><s></code> frase <code></s></code> testo della capacità (con lo stato) <code></s></code>
15	Modello → Motore	uscite <code>score</code> , <code>types</code>	13 punteggi + probabilità dei tipi (DEVICE)
16	Motore	politica e regole	<code>light.turn_on@area:cucina:light</code> , $p = 0,9975$, rischio basso (soglia 0,85) → <code>EXECUTE</code>
17	Motore → Server	Python: oggetto <code>Decision</code>	policy, reason «ok», capacità, 3 entità, confidenza
18	Server	<code>reply()</code>	risposta «Fatto.»
19	Server → Adattatore	Python: <code>execute()</code>	la decisione
20	Adattatore → Core	HTTP POST <code>.../api/services/light/turn_on</code>	<code>{"entity_id": ["light.pendente_tavolo", "light.faretti_isola", "light.striscia_mensola"]}</code>
21	Core	integrazioni delle luci	le 3 luci si accendono
22	Core → Adattatore	HTTP 200	stati cambiati
23	Server → Integrazione	HTTP 200, JSON	<code>{"decision": {"policy": "EXECUTE", ...}, "reply": "Fatto.", "execution": {"executed": true, "service": "light.turn_on", ...}}</code>
24	Integrazione → Core	evento sul bus di HA	<code>edgedecision_decision</code> : testo, stanza, decisione, tempi
25	Integrazione → Assist	Python: <code>ConversationResult</code>	testo da pronunciare «Fatto.», nessuna domanda aperta
26	Assist	TTS	audio di «Fatto.»
27	Assist → Voice PE	audio (ESPHome)	l'altoparlante riproduce la risposta

7.2 Secondo esempio: due comandi in una frase

Con «accendi luce salotto e spegni la tv della taverna» (Figura 7) il software valuta prima la frase intera; poiché contiene due verbi di comando (accendi, spegni), la divide sulla congiunzione «e» e decide ciascuna parte con le sue due passate: sei passate del modello in tutto, circa il triplo del lavoro di un comando semplice. «Salotto» non è il nome di nessuna stanza della casa di prova, ma il modello lo associa a «Salone» (probabilità 0,9977); la seconda parte sceglie la TV della taverna (0,9999). Entrambe superano le soglie, la decisione complessiva è `EXECUTE` con due parti e il software esegue due chiamate di servizio, `light.turn_on` su Lampada ad arco e Faretto salotto e `media_player.turn_off`

sulla TV della taverna, poi compone un'unica risposta: «Accendo le luci in Salone e spengo TV taverna.». Se una delle due parti fosse incerta, il sistema eseguirebbe quella chiara e farebbe una domanda solo sull'altra.

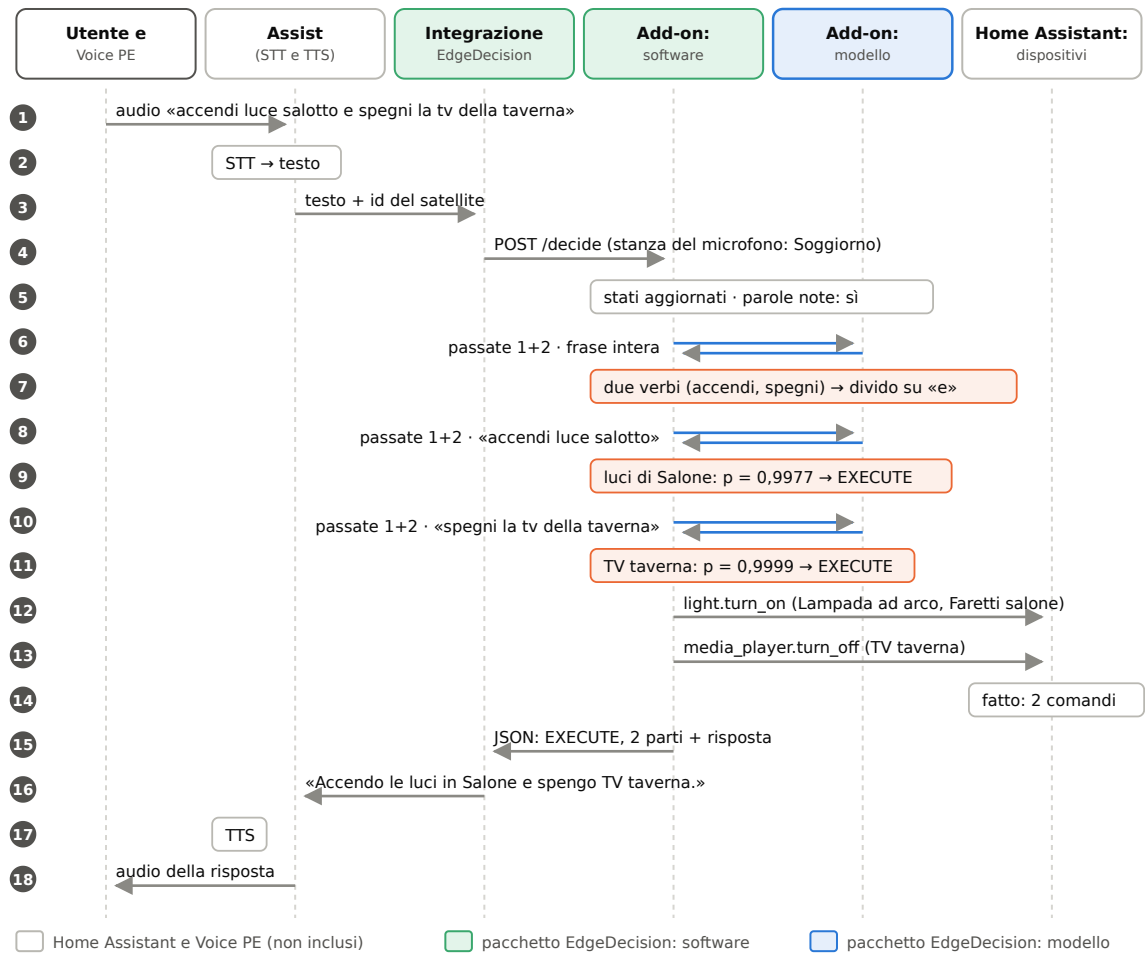


Figura 7. Flusso di un comando doppio. Ogni doppia freccia blu indica le due passate del modello (vettore della frase e punteggio di 13 coppie) su un testo: la frase intera e poi ciascuna delle due parti.

La Figura 8 mostra lo stesso comando doppio dentro l'add-on, messaggio per messaggio: tre coppie di passate del modello, due decisioni della politica e due chiamate di servizio, raccolte in un'unica risposta HTTP (Tabella 5).

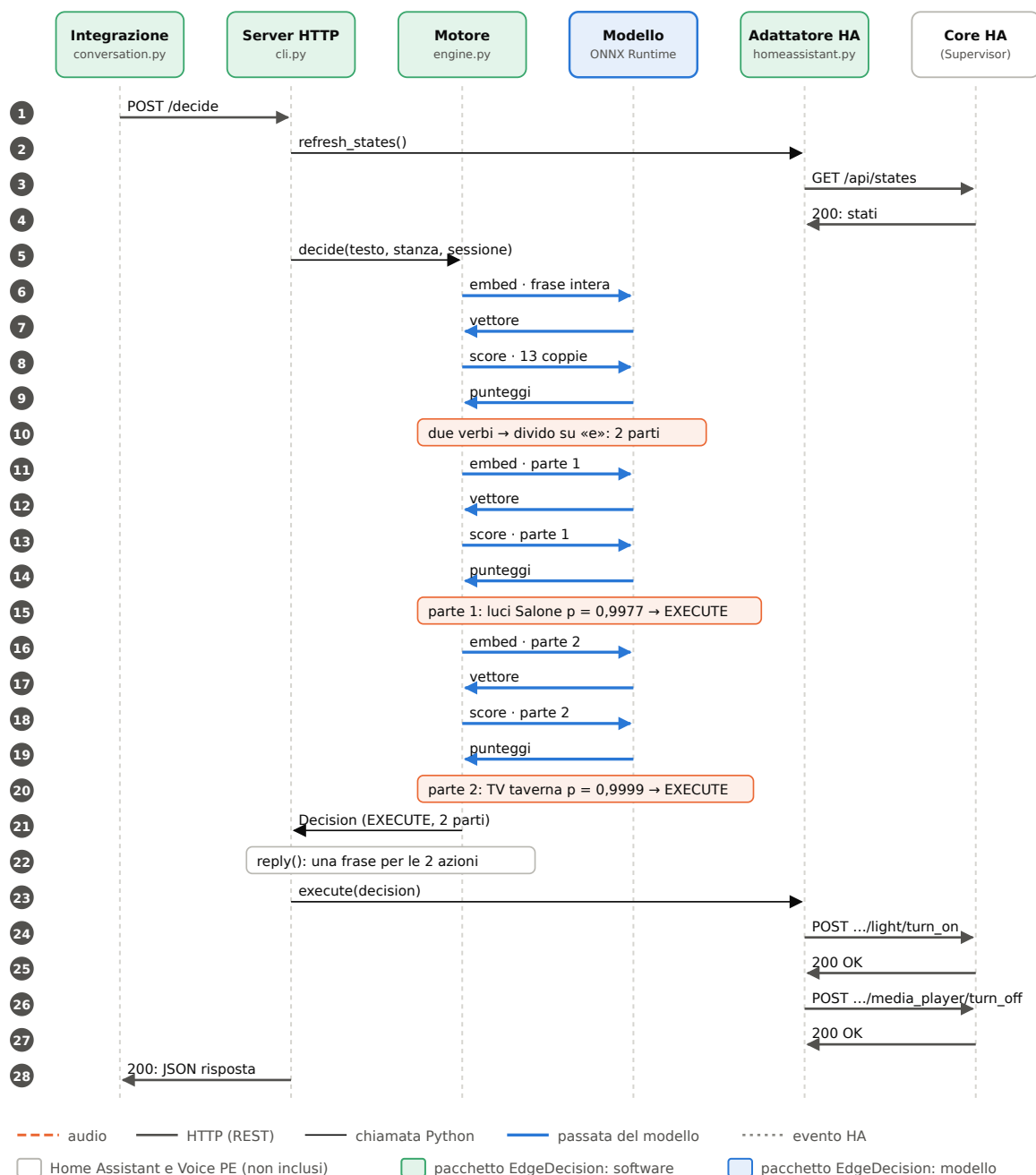


Figura 8. Il comando doppio all'interno dell'add-on. Stessa legenda della Figura 6; la parte vocale (Voice PE, STT, TTS) è identica al primo esempio e non è ripetuta.

Tabella 5. Contenuto dei messaggi della Figura 8 (i passaggi uguali al primo esempio sono raggruppati).

#	Da → a	Canale	Contenuto
1	Integrazione → Server	HTTP POST /decide	{ "text": "accendi luce salotto e spegni la tv della taverna", "context_area": "soggiorno", "session": "<id satellite>", "execute": true, "lang": "it" }
2-5	Server ↔ Adattatore ↔ Core; Server → Motore	come nel primo esempio	stati aggiornati, poi engine.decide()
6-9	Motore ↔ Modello	2 passate	frase intera, valutata per prima; con due verbi di comando il sistema la divide comunque
10	Motore	divisione	due verbi di comando (accendi, spegni) → parti «accendi luce salotto» e «spegni la tv della taverna»
11-15	Motore ↔ Modello	2 passate + politica	parte 1 → light.turn_on@area:salone:light, p = 0,9977 → EXECUTE

#	Da → a	Canale	Contenuto
16-20	Motore ↔ Modello	2 passate + politica	parte 2 → media_player.turn_off@media_player.tv_taverna, p = 0,9999 → EXECUTE
21	Motore → Server	Python: <code>Decision</code>	policy EXECUTE, reason «multi_intent», 2 parti
22	Server	<code>reply()</code>	«Accendo le luci in Salone e spengo TV taverna.»
24	Adattatore → Core	HTTP POST .../api/services/light/turn_on	<code>{"entity_id": ["light.lampada_arco", "light.faretti_salone"]}</code>
26	Adattatore → Core	HTTP POST .../api/services/media_player/turn_off	<code>{"entity_id": ["media_player.tv_taverna"]}</code>
28	Server → Integrazione	HTTP 200, JSON	<code>{"decision": {"policy": "EXECUTE", "reason": "multi_intent", "parts": [...]}, "reply": "Accendo le luci in Salone e spengo TV taverna.", "execution": {"parts": [{...}, {...]}}</code>

8. Un esempio passo per passo: dalle parole alla decisione

Questa sezione segue una frase reale, «accendi la tv» detta in taverna nella casa di prova (237 capacità), dal testo alla decisione. Tutti i numeri sono quelli prodotti dal modello a 6 strati del ciclo 4. Nel caso normale il modello viene eseguito **due volte**: una passata breve sulla sola frase, per il recupero dei candidati, e una passata sulle 13 coppie (frase, candidato), per la decisione.

8.1 Dalle parole ai numeri (passata 1)

Il modello non legge lettere ma numeri (Figura 9). La frase, con la stanza del microfono in coda («accendi la tv || user is in: Taverna»), viene spezzata dal tokenizer SentencePiece in 17 pezzi: parole frequenti restano intere («_la», «_tv»), quelle rare si dividono («_ac» + «cendi»); il segno _ indica l'inizio di una parola, <s> e </s> l'inizio e la fine del testo. Ogni pezzo ha un numero nel vocabolario (24.567 voci dopo la potatura) e ogni numero seleziona una riga della tabella degli embedding: 17 vettori da 384 numeri. I 6 strati dell'encoder li trasformano: in ogni strato, tramite l'attenzione, ciascun pezzo combina le informazioni di tutti gli altri, così che alla fine «tv» «sa» di essere l'oggetto di «accendi» e di trovarsi in taverna. La media dei 17 vettori, proiettata su 256 dimensioni e normalizzata a lunghezza 1, è il *vettore della frase*.

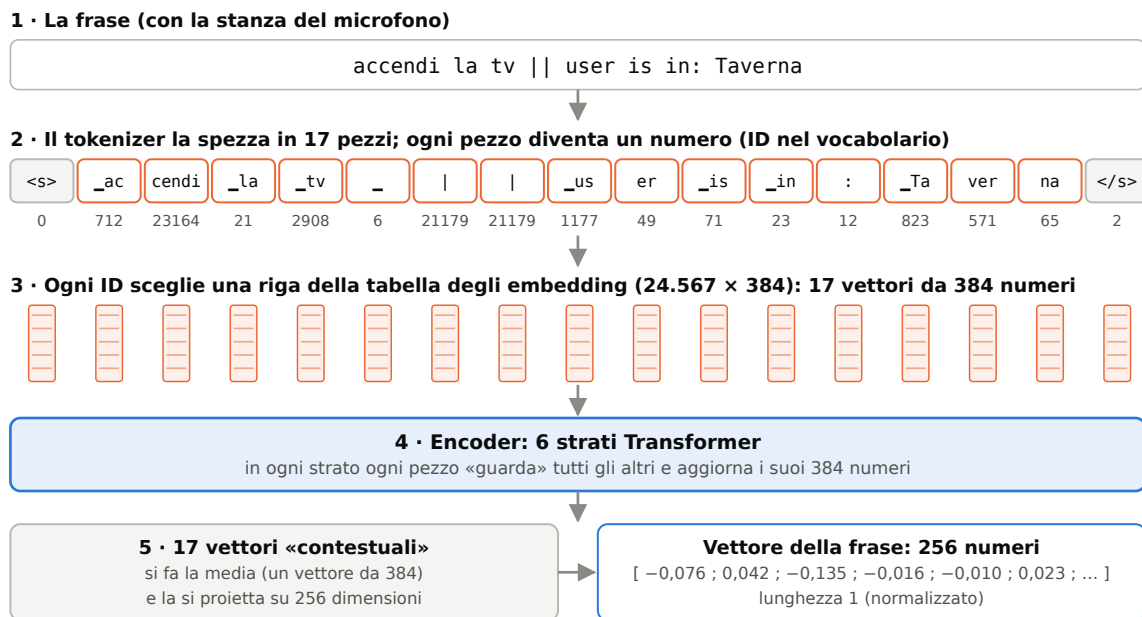


Figura 9. Passata 1: dalla frase al vettore di 256 numeri usato per il recupero. Pezzi e ID sono quelli reali del tokenizer; i primi sei valori del vettore sono quelli reali prodotti dal modello. Le colonne del passo 3 sono schematiche (ciascuna contiene 384 numeri).

8.2 Recupero dei 12 candidati

Il recupero (Figura 10) confronta la frase con tutte le 237 capacità in due modi e ne somma i risultati.

- **Per significato.** Il testo di ogni capacità (per esempio «media player turn on: ... || target: TV taverna ... || area: Taverna», senza lo stato) è stato trasformato con la stessa passata 1 in un vettore di 256 numeri *una volta sola*, all'avvio e ogni volta che il catalogo cambia (circa 3 s per 237 capacità sulla macchina di prova). Poiché tutti i vettori hanno lunghezza 1, un solo prodotto matrice-vettore dà le 237 similarità coseno: 0,88 per la TV della taverna, 0,77 per quella del salone, 0,14 per la tapparella del salone.
- **Per lettere.** Ogni capacità ha anche una «scheda» di parole: nome e alias del dispositivo, nome della stanza e parole chiave dell'azione in italiano, inglese e spagnolo (per la TV della taverna: «TV taverna Taverna accendi accendere ... turn on switch on ... tv televisione ...»). La frase e le schede sono scomposte in n-grammi di 3 e 4 caratteri e confrontate con un indice TF-IDF, che dà più peso agli n-grammi rari. Sui 32 n-grammi di «accendi la tv Taverna», 29 compaiono nella scheda della TV della taverna. Questo confronto non usa il modello e resiste agli errori di trascrizione: «a cendi» condivide ancora 7 dei 13 n-grammi di «accendi» («cen», «end», «ndi»...).

Il punteggio totale è $0,7 \times \text{significato} + 0,3 \times \text{lettere}$ (il secondo diviso per il valore massimo, così va da 0 a 1). Si tengono le 12 capacità con il totale più alto e si aggiunge sempre `NO_ACTION`. Le altre 225 capacità non vengono valutate dal modello. Se la capacità giusta resta fuori dalle 12 il modello non può sceglierla, ma nemmeno inventarne un'altra: al massimo sceglie `NO_ACTION` o chiede. Succede di rado: la capacità giusta è tra le 12 nel 99,9% delle frasi di validazione e nel 100% delle frasi gold.



Figura 10. Recupero dei candidati per «accendi la tv» in taverna, con i valori reali. A sinistra il confronto per significato, che usa il vettore della Figura 9; a destra quello per lettere, che non usa il modello. Il punto (·) negli n-grammi indica l'inizio di una parola.

8.3 Punteggio e decisione (passata 2)

Per ciascuno dei 13 candidati si forma una coppia di testi, `<s> frase </s></s> capacità </s>`, che per la TV della taverna è lunga 61 pezzi perché contiene anche lo stato attuale («state: off»). Le 13 coppie entrano nel modello insieme, come un unico lotto: è la passata 2 della Figura 1. Per ogni coppia la testa di punteggio legge il vettore del primo pezzo, `<s>`, che dopo i 6 strati riassume il rapporto tra frase e capacità, e produce un numero: 10,03 per la TV della taverna, -0,17 per quella del salone, -3,45 per NO_ACTION. La coppia con NO_ACTION fornisce anche il tipo di richiesta (qui DEVICE). La softmax con temperatura 1,31 dà le probabilità (0,9995 per la TV della taverna), la politica controlla soglie e margine per il rischio basso e le regole deterministiche confermano che la frase indica un solo dispositivo della stanza: EXECUTE.

8.4 Quante passate e quanto tempo

La Figura 11 mostra dove va il tempo di una decisione, misurato su una macchina di prova con 2 core (mediana di 25 ripetizioni: 280 ms in totale). La passata 2 occupa il 96% del tempo perché elabora 13 testi lunghi; la passata 1 lavora su un solo testo corto (circa 3%); recupero, politica e regole, che non usano il modello, pesano meno dell'1%. Sul Raspberry Pi 5 la stessa decisione richiede circa 150 ms e sul PC di addestramento 39 ms; le quote non sono state misurate separatamente su quei dispositivi, ma il lavoro svolto è lo stesso. Il numero di passate è fisso per ogni tipo di frase e non dipende dalla lunghezza della risposta (Tabella 6).

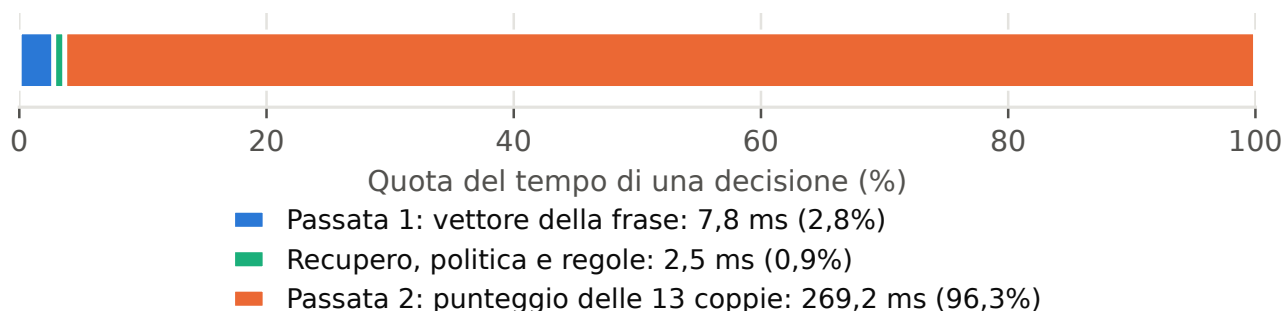


Figura 11. Tempo di una decisione per «accendi la tv» in taverna, diviso tra le due passate del modello e il resto (macchina di prova con 2 core, mediana di 25 ripetizioni).

Tabella 6. Passate del modello per frase, contate eseguendo il sistema sulla casa di prova.

Caso	Esempio	Passate sulla frase	Passate da 13 coppie
Comando semplice	«accendi la tv», in taverna	1	1
Nessun dispositivo adatto nella stanza del microfono: si cerca in tutta la casa	«accendi la tv», in camera di Marco (poi chiede quale TV)	2	2
Due comandi: la frase intera e poi ciascuna parte	«spegni le luci della cucina e accendi la tv della taverna»	3	3
Modello del ciclo 5: etichette delle parole	qualsiasi frase	+1	—
Una volta sola, non per frase	avvio o catalogo cambiato: i 237 testi delle capacità		circa 3 s

9. Il modello

Il modello si chiama **e5s-6L**: «e5s» indica il modello da cui deriva, *multilingual-e5-small*, e «6L» i suoi 6 strati. La rete (Figura 12) è un encoder Transformer condiviso con quattro teste leggere. La testa di punteggio e quella di tipo leggono il token iniziale (CLS) di una coppia di testi; la proiezione degli embedding usa la media dei token di un testo singolo e produce un vettore normalizzato di 256 dimensioni; la testa di etichette (dal quinto ciclo) classifica ogni parola della frase come verbo o comando (CMD), dispositivo o stanza (OBJ), valore (VAL), separatore tra due comandi (SEP) o altro (O). Tutte le teste sono nello stesso grafo ONNX, quindi recupero, punteggio e divisione dei comandi usano lo stesso file.

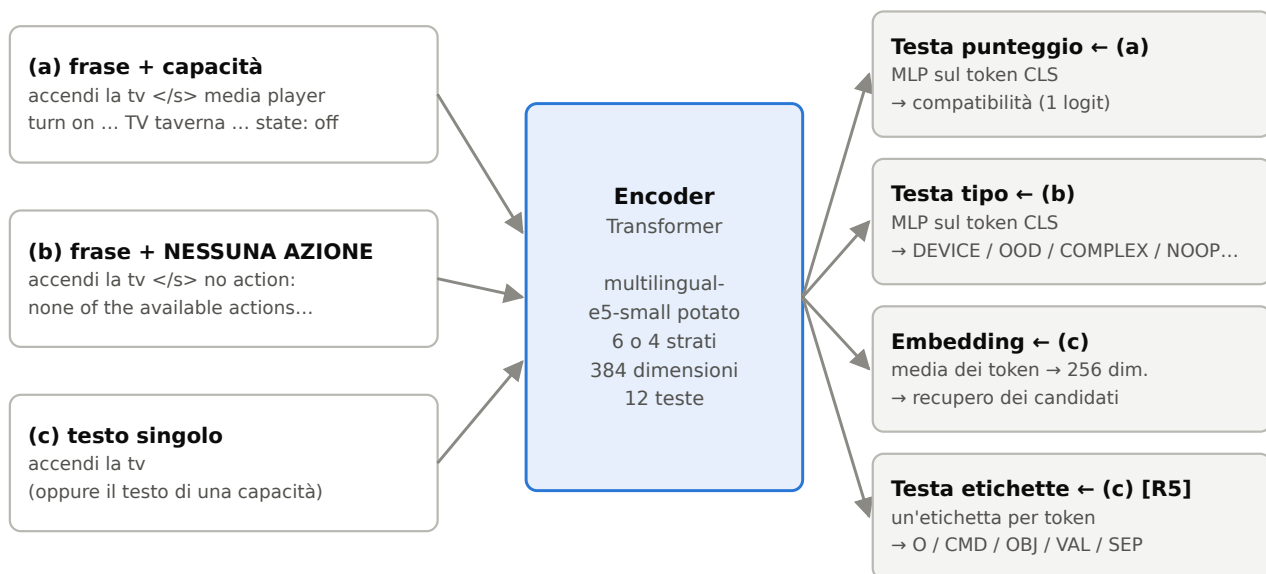


Figura 12. La rete di EdgeDecision: un encoder condiviso e quattro teste. (a) e (b) sono coppie di testi separate dal token </s>; (c) è un testo singolo. La testa di etichette è presente dal quinto ciclo (R5).

10. Dati

Non esiste un corpus pubblico di comandi domotici etichettati con il catalogo della casa, quindi tutti i dati di addestramento sono sintetici (Figura 13).

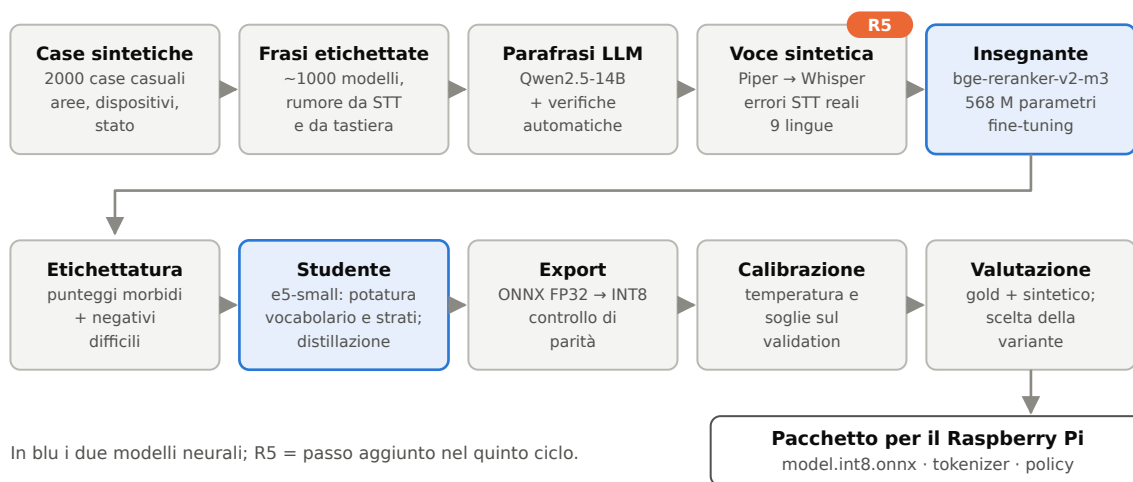


Figura 13. La procedura di addestramento. Tutto gira su un PC con una RTX 3090; al Raspberry Pi arriva solo il pacchetto finale.

Case sintetiche. Un generatore crea case casuali: stanze con nomi e alias in più lingue, dispositivi scelti con probabilità realistiche per ogni stanza (luci, tapparelle e tende, clima, TV e casse, ventilatori, prese, serrature, robot aspirapolvere, allarmi, sensori, scene), con funzioni e stati coerenti. Circa il 10% dei dispositivi di alcuni tipi riceve un nome tecnico («Shelly 2»), che chi parla deve pronunciare.

Frase etichettate. Circa 1000 modelli di frase producono comandi, domande sullo stato, richieste ambigue, valori mancanti o fuori scala, negazioni, richieste fuori tema, richieste complesse e dispositivi assenti, ciascuno con la decisione attesa. Un generatore di rumore simula errori tipici del riconoscimento vocale e della tastiera. Circa un quarto delle frasi del test sintetico usa modelli di frase mai visti in addestramento.

Parafrasi con LLM. Un LLM locale (Qwen2.5-14B-Instruct, tramite Ollama) riscrive frasi-seme in modo più naturale. Ogni parafrasi deve superare controlli automatici: gli stessi valori numerici estratti dal parser di runtime, la stessa stanza (né persa né inventata) e un giudizio a scelta multipla dello stesso LLM tra la capacità corretta e capacità simili. Nel primo ciclo, da circa 4000 semi sono state generate 31.753 parafrasi, di cui 21.602 (68%) accettate.

Insieme gold. 227 frasi scritte a mano in italiano (106), inglese (59) e spagnolo (62) su una casa dimostrativa, mai usate in addestramento né per scegliere iperparametri, misurano il comportamento su formulazioni realistiche.

Tabella 7. Dimensioni dei dati. Cicli 1–4: 3 lingue (IT, EN, ES). Ciclo 5: 9 lingue; le case dei cicli precedenti sono riusate e si aggiungono 350 case per ciascuna lingua nuova.

	Cicli 1–4	Ciclo 5
Case di addestramento / validazione / test	2000 / 150 / 150	4100 / 300 / 300
Frazi da modelli (addestramento)	100.000	205.000
Parafrasi LLM e frasi generate dal LLM	24.308	50.648
Trascrizioni della voce sintetica	—	9.556
Totale esempi di addestramento etichettati	124.308	265.204
Test sintetico (case mai viste)	7.500	15.000
Insieme gold scritto a mano	227	227

11. Come abbiamo ottenuto il modello e5s-6L

Questa sezione descrive come si passa da due modelli pre-addestrati generici a un modello di 24,3 MB che gira su un Raspberry Pi. Il modello del Raspberry Pi non è addestrato da zero: è `multilingual-e5-small` trasformato in cinque passi (Figura 14). I primi due lo rimpiccioliscono, il terzo gli dà le uscite che servono per decidere, il quarto gli insegna il compito imitando un modello molto più grande, il quinto lo comprime e ne tara le probabilità. La Figura 15 mostra quanto pesa il modello dopo ogni passo; la Tabella 8 riassume i modelli di partenza.

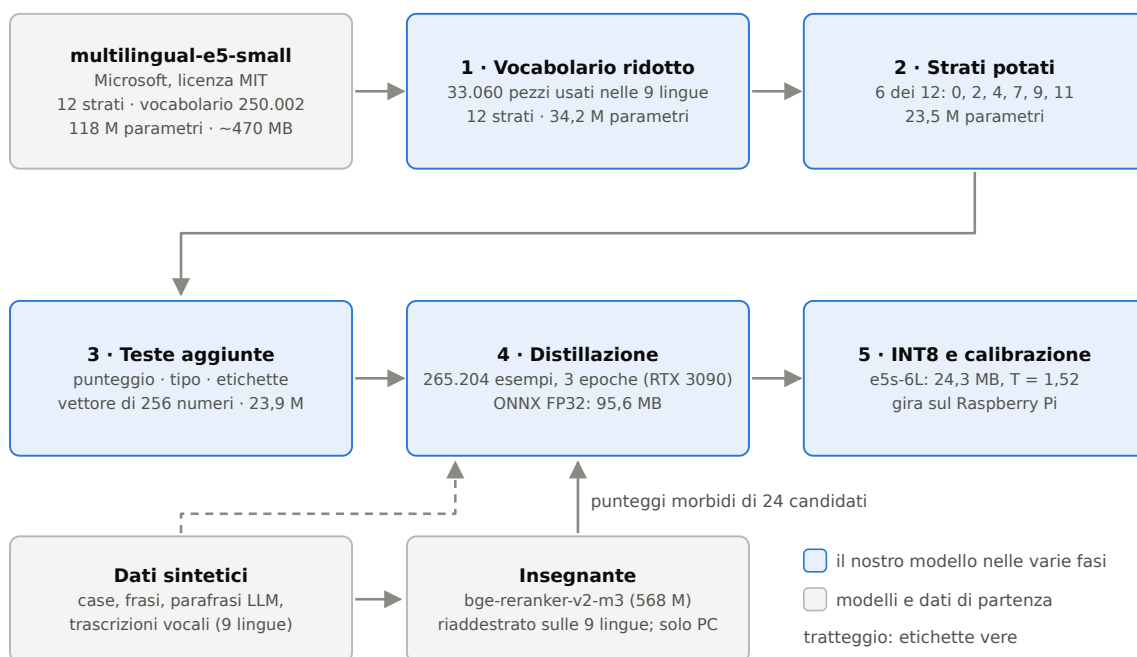


Figura 14. Da multilingual-e5-small a e5s-6L (ciclo 5). In blu il nostro modello nelle varie fasi, in grigio modelli e dati di partenza. L'insegnante e i dati sintetici servono solo durante l'addestramento, sul PC; sul Raspberry Pi arriva solo il risultato del passo 5. Lo studente impara dai punteggi dell'insegnante (freccia piena) e dalle etichette vere dei dati (tratteggio).

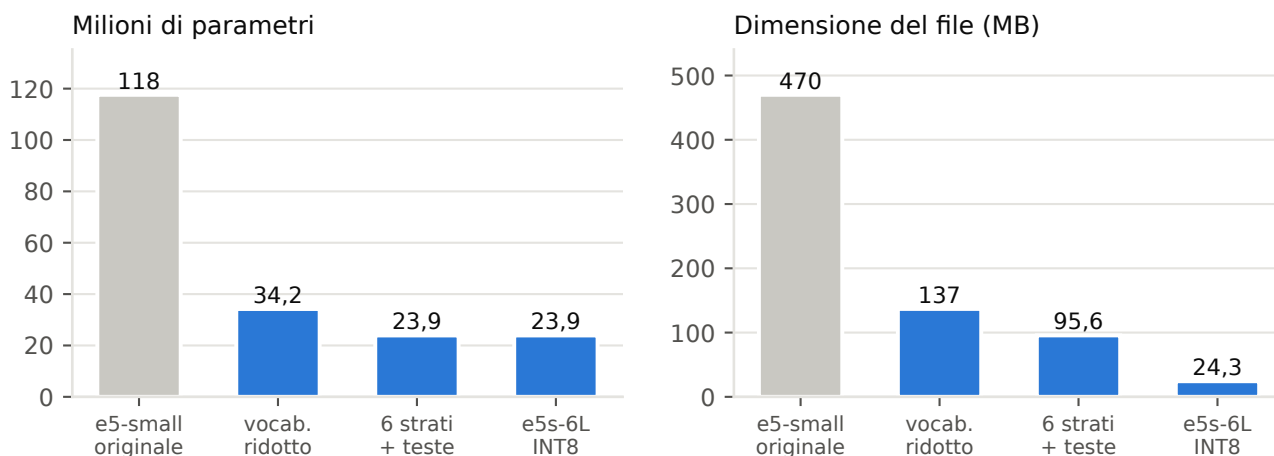


Figura 15. Parametri e dimensione del file dopo ogni passo. La riduzione del vocabolario elimina da sola il 71% dei parametri; la potatura degli strati toglie un altro terzo; la quantizzazione INT8 non cambia il numero di parametri ma divide il file per quasi 4. Le dimensioni in FP32 sono quelle dei file reali, tranne il modello a 12 strati con vocabolario ridotto, stimato come 4 byte per parametro.

Tabella 8. I modelli di partenza.

	Insegnante	Studente
Modello	BAAI/bge-reranker-v2-m3	intfloat/multilingual-e5-small
Tipo di partenza	cross-encoder per ri-ordinamento	encoder per embedding di frasi
Architettura	XLNet-RoBERTa large: 24 strati, 1024 dim.	12 strati, 384 dim., 12 teste
Parametri	568 M	118 M (di cui 96 M di vocabolario)
Vocabolario	250.002 pezzi	250.002 pezzi
Ruolo	solo durante l'addestramento, sul PC	diventa il modello del Raspberry Pi

11.1 Preparazione: addestramento dell'insegnante

L'insegnante è stato riaddestrato per un'epoca sul compito di EdgeDecision (punteggio a lista più tipo di richiesta), su 90.000 esempi, con gruppi di 16 candidati, bf16 e gradient checkpointing per stare nei 24 GB della GPU. Sul validation mette al primo posto la risposta corretta nel 98,9% dei casi e riconosce il tipo di richiesta nel 100%. I cicli 2–4 hanno riusato lo stesso insegnante; il ciclo 5 lo ha riaddestrato su 130.000 esempi in 9 lingue (98,5% e 99,5%).

11.2 Preparazione: etichettatura

Per ogni esempio di addestramento l'insegnante valuta la frase con NESSUNA AZIONE, con le capacità corrette e con altre 32 capacità simili per testo o per struttura, e conserva i punteggi di 24 candidati. Questi *punteggi graduati (soft)* dicono allo studente non solo qual è la risposta giusta ma anche quali alternative sono quasi giuste, e i candidati con punteggio alto ma sbagliati diventano *negativi difficili*. Nel primo ciclo l'insegnante è d'accordo con l'etichetta vera sul 97,4% degli esempi di addestramento; sugli altri lo studente impara solo dall'etichetta vera.

11.3 Passo 1: riduzione del vocabolario

In multilingual-e5-small la tabella degli embedding (250.002 pezzi $\times$ 384 dimensioni) contiene 96 dei 118 milioni di parametri, perché serve circa 100 lingue. Teniamo solo: i pezzi che compaiono nel corpus di addestramento, i 20.000 pezzi più probabili in alfabeto latino, per coprire parole nuove come i nomi dei dispositivi, i singoli caratteri e i token speciali. Con le tre lingue dei cicli 1–4 il vocabolario scendeva a 24.567 pezzi; con le nove lingue del ciclo 5 sale a 33.060. Il modello a 12 strati passa così da 118 a 34,2 milioni di parametri (34,5 con le teste del passo 3, Figura 16). La tokenizzazione non cambia: nel controllo del ciclo 1, su 3006 frasi le sequenze prodotte dal vecchio e dal nuovo vocabolario coincidono tutte.

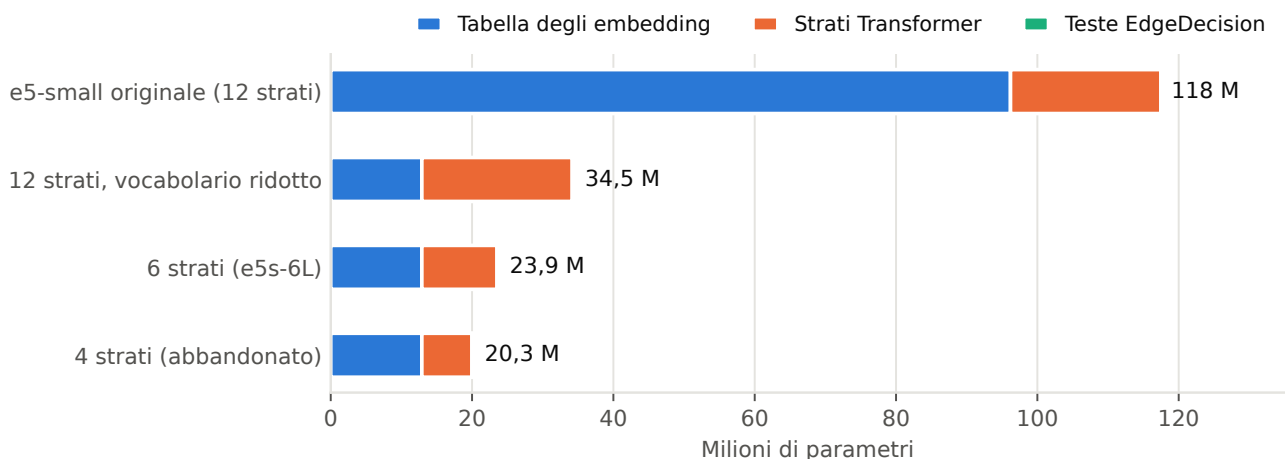


Figura 16. Parametri per componente, con il vocabolario del ciclo 5 (le teste di EdgeDecision, 0,32 M, sono troppo piccole per essere visibili). La riduzione del vocabolario da sola divide il modello per 3,4; la potatura degli strati riduce solo la parte Transformer. La variante a 4 strati è stata addestrata fino al ciclo 5 e poi abbandonata.

11.4 Passo 2: potatura degli strati

Dall'encoder a 12 strati si tengono k strati distribuiti uniformemente, sempre compresi il primo e l'ultimo (Figura 17). Il primo strato legge i token e l'ultimo produce le rappresentazioni che le teste si aspettano; quelli intermedi sono in parte ridondanti. Con 6 strati su 12 il modello scende da 34,2 a 23,5 milioni di parametri. Il modello potato perde accuratezza, che recupera con l'addestramento del passo 4.

11.5 Passo 3: le teste di decisione

Sopra l'encoder si aggiungono le quattro teste della Figura 12: punteggio di una coppia frase-capacità, tipo di richiesta (6 classi), etichetta di ogni parola (5 classi) e proiezione a un vettore di 256 numeri per il recupero. Insieme valgono 0,32 milioni di parametri, meno del 2% del modello, che arriva così a 23,9 milioni. Le teste partono da pesi casuali: tutto il loro sapere viene dal passo 4.



Figura 17. Strati dell'encoder originale mantenuti nelle varianti a 6 strati (0, 2, 4, 7, 9, 11) e a 4 strati (0, 4, 7, 11).

11.6 Passo 4: addestramento per distillazione

Ogni variante (12, 6 o 4 strati) viene addestrata sugli esempi etichettati con la perdita combinata (punteggio, tipo di richiesta, distillazione, recupero ed etichette): 3 epoche (4 nel ciclo 1 e per la 4 strati), batch di 16 gruppi da 16 candidati, learning rate $6 \cdot 10^{-5}$ per l'encoder e $5 \cdot 10^{-4}$ per le teste, bf16. Nel ciclo 1 la 12 strati raggiunge il 99,27% di primo posto corretto sul validation e la 6 strati il 99,03%: dimezzare gli strati costa solo 0,24 punti. Nel ciclo 5 lo studente a 6 strati è stato addestrato su 265.204 esempi in nove lingue, per 3 epoche.

11.7 Passo 5a: esportazione e quantizzazione

Il modello viene esportato in ONNX (FP32), ottimizzato e quantizzato in INT8 con quantizzazione dinamica dei pesi. Un controllo di parità confronta i punteggi con quelli di PyTorch: in FP32 la differenza massima resta sotto 10^{-5} ; in INT8 la correlazione con i punteggi originali resta 0,9995 per la 6 strati del ciclo 4 e 0,9990 per quella del ciclo 5. Nel ciclo 5 il file della 6 strati scende da 95,6 a 24,3 MB.

11.8 Passo 5b: calibrazione e scelta della variante

Su 4000 esempi di validazione, usando il modello INT8 e il recupero reale, si stimano la temperatura e poi le soglie della Tabella 2, cercando la combinazione che massimizza l'accuratezza con una precisione delle esecuzioni di almeno il 99%. Infine si sceglie la variante: tra quelle con latenza p95 non superiore a 120 ms sul PC (4 thread) e precisione delle esecuzioni gold almeno del 98%, vince quella con l'accuratezza gold più alta. Nel ciclo 5 la temperatura stimata è 1,52; l'obiettivo del 99% di precisione sul validation non è stato raggiunto (98,5%) e sono rimaste le soglie del ciclo 4. Dal ciclo 5 si addestra solo la variante a 6 strati (vedi «Perché solo 6 strati»).

Tabella 9. Le varianti dello studente (ciclo 5; per la 12 strati, ciclo 2 con il vocabolario di allora). Latenze sul PC di addestramento (CPU, 4 thread) sulle frasi gold, compreso il recupero.

Variante	Strati	Parametri	ONNX FP32	ONNX INT8	Latenza p50 / p95
e5s-12L (ciclo 2)	12	31,2 M	125,2 MB	32,0 MB	80,8 / 131,3 ms
e5s-6L	6	23,9 M	95,6 MB	24,3 MB	41,4 / 66,6 ms
e5s-4L (abbandonata)	4	20,3 M	81,4 MB	20,6 MB	29,3 / 47,8 ms

12. Risultati

Le metriche principali sono l'**accuratezza della decisione** (la decisione e, per EXECUTE, la capacità coincidono con quelle attese) e la **precisione delle esecuzioni** (la frazione di EXECUTE che eseguono l'azione giusta): la seconda misura il rischio, perché un'esecuzione sbagliata cambia davvero qualcosa in casa.

12.1 Compromesso tra accuratezza e latenza

Il ciclo 2 è l'unico in cui sono state addestrate tutte e tre le varianti (Figura 18). La 12 strati è la più accurata (97,8% sul gold) ma la sua latenza p95, 131 ms sul PC, supera il limite; la 6 strati perde 0,9 punti e dimezza la latenza; la 4 strati è la più veloce ma perde altri 1,3 punti. Dal ciclo 3 in poi la 12 strati non è più stata addestrata.

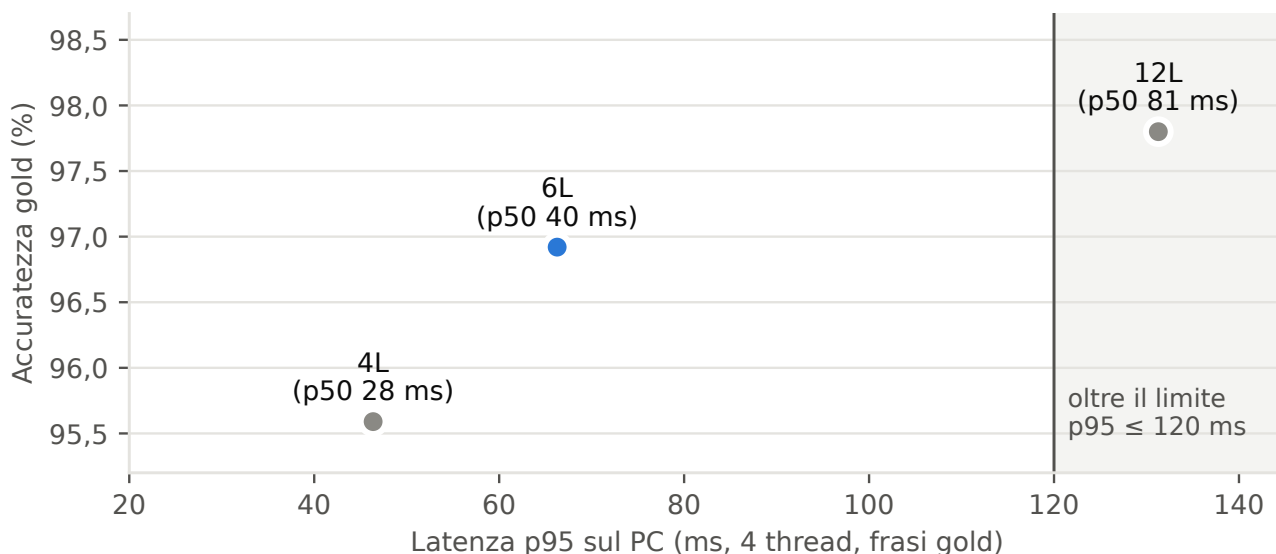


Figura 18. Accuratezza sulle 227 frasi gold rispetto alla latenza p95 sul PC, ciclo 2. La zona grigia supera il limite di 120 ms; tra parentesi la latenza mediana.

12.2 Miglioramento tra i cicli

Ogni ciclo è partito dall'analisi degli errori del precedente e ha modificato i dati, non il modello. Ciclo 2: etichette corrette ed esempi aggiunti dove il ciclo 1 sbagliava. Ciclo 3: dispositivi il cui nome contiene il nome generico («Ventilatore a soffitto») e richieste di contenuti multimediali. Ciclo 4: dispositivi chiamati per nome, microfono in una stanza diversa da quella del comando, possibilità di includere la propria casa reale. Sul gold la 6 strati passa dal 96,5% al 97,8% e le esecuzioni sbagliate da 3 a 0 (Figura 19, Tabella 10). Il ciclo 5 aggiunge sei lingue: sul gold, che è solo in italiano, inglese e spagnolo, la 6 strati scende al 96,9%, sempre senza esecuzioni sbagliate, e torna al 97,8% con le correzioni del software descritte più avanti. L'accuratezza sul test sintetico scende invece leggermente tra i cicli perché il test viene rigenerato a ogni ciclo e include i casi difficili appena introdotti: non è confrontabile tra cicli, il gold sì.

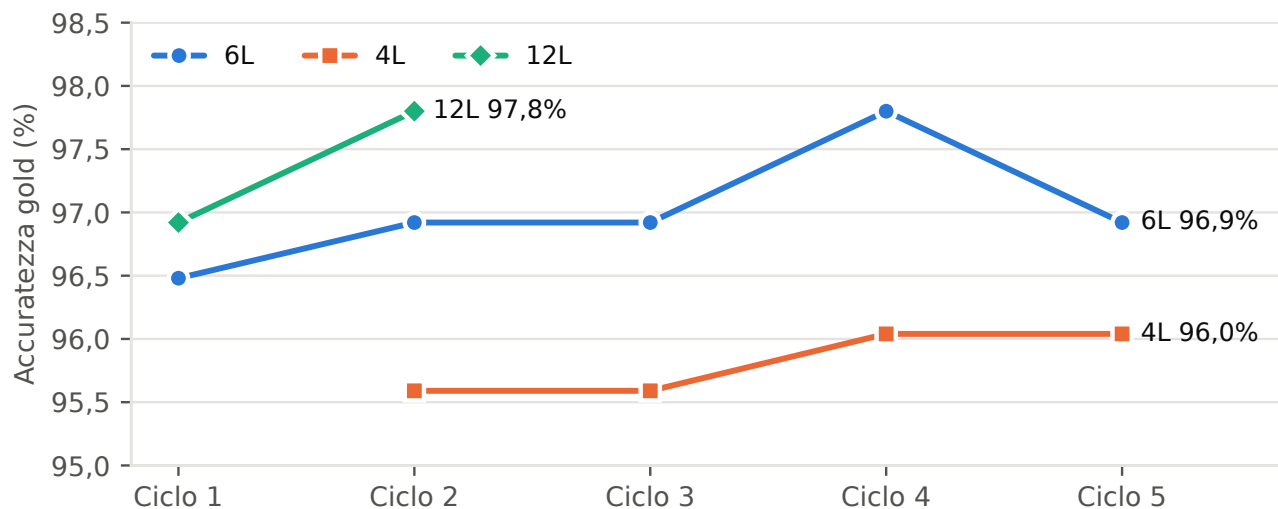


Figura 19. Accuratezza sulle frasi gold nei cinque cicli. La 4 strati è stata introdotta nel ciclo 2 e abbandonata dopo il ciclo 5; la 12 strati non è stata più addestrata dopo il ciclo 2. Il calo del ciclo 5 è il prezzo delle sei lingue nuove, misurato con il software di allora.

Tabella 10. Risultati per ciclo e variante. Gold: 227 frasi scritte a mano; sintetico: 7500 frasi su 150 case mai viste nei cicli 1–4, 15.000 su 300 case in 9 lingue nel ciclo 5 (rigenerato a ogni ciclo). Prec. = precisione delle esecuzioni; err. = esecuzioni sbagliate. Evidenziata la variante in uso sul Raspberry Pi.

Ciclo	Variante	Gold acc.	Gold prec.	Gold err.	Sint. acc.	Sint. prec.
1	12L	96,9%	99,4%	1	95,5%	97,2%
1	6L	96,5%	98,1%	3	94,8%	96,9%
2	12L	97,8%	98,7%	2	95,7%	98,2%
2	6L	96,9%	99,3%	1	93,8%	97,3%
2	4L	95,6%	98,7%	2	93,6%	97,5%
3	6L	96,9%	100%	0	93,7%	97,5%
3	4L	95,6%	100%	0	92,7%	97,8%
4	6L	97,8%	100%	0	92,8%	97,6%
4	4L	96,0%	100%	0	91,0%	97,1%
5	6L	96,9%	100%	0	92,9%	97,5%
5	4L	96,0%	99,3%	1	92,5%	96,8%

La Tabella 11 mostra le decisioni della variante in uso. I 5 errori sul gold sono tutti prudenti: comandi eseguibili che portano a una domanda invece che all'esecuzione («alza la luminosità del lampadario», «turn on the floor lamp», «activa el riego»...); nessuna azione sbagliata viene eseguita. Per lingua l'accuratezza è 98,1% in italiano, 98,3% in inglese e 96,8% in spagnolo.

Tabella 11. Matrice di confusione sulle 227 frasi gold (6 strati, ciclo 4). Righe: decisione attesa; colonne: decisione presa.

Attesa \ presa	EXECUTE	CLARIFY	REJECT	ESCALATE
EXECUTE	146	5	0	0
CLARIFY	0	20	0	0
REJECT	0	0	23	0
ESCALATE	5	0	0	28

Le 5 frasi della riga ESCALATE decise come EXECUTE non sono errori: sono comandi multipli («accendi X e chiudi Y») che l'insieme gold etichetta come ESCALATE, ma per i quali la valutazione accetta anche l'esecuzione separata di ogni parte, purché ciascuna sul dispositivo giusto, come in questi 5 casi. Per questo l'accuratezza è $222/227 = 97,8\%$ e non la somma della diagonale. Sul validation, invece, la calibrazione non ha raggiunto l'obiettivo del 99% di precisione: con le soglie scelte la precisione è 98,5% e il 91,4% dei comandi eseguibili viene eseguito.

12.3 Prova sul campo: Raspberry Pi 5 e Home Assistant

Per una prova indipendente dai dati di addestramento abbiamo creato in Home Assistant una casa di prova con 51 dispositivi virtuali (luci, tapparelle, termostati, due TV, serrature, robot, allarme, sensori, scene), con nomi mai usati nel generatore. EdgeDecision, variante 6 strati del ciclo 4, gira come add-on su un Raspberry Pi 5 con Home Assistant OS. I risultati sono nella Tabella 12.

Tabella 12. Prova sul Raspberry Pi 5 (6 strati, ciclo 4).

Prova	Risultato
Decisioni corrette su 170 frasi (130 IT, 21 EN, 19 ES; nessuna esecuzione)	94,1% (160/170)
Esecuzioni sbagliate / precisione delle esecuzioni	1 / 99,2%
Scenari eseguiti sui dispositivi virtuali, con dialoghi e conferme	20 / 20
Prove attraverso Assist (agente di conversazione)	5 / 5
Tempo di decisione sul Pi, mediana	≈ 150 ms

L'unica esecuzione sbagliata è istruttiva: «chiudi il lucernario della mansarda» ha spento la luce della mansarda. «Lucernario» non compare nei nomi dei dispositivi e somiglia a «luce»; il modello ha scelto il dispositivo della stanza giusta con l'azione più vicina. Gli altri errori sono prudenti: domande invece di esecuzioni, o rifiuti per dispositivi che il modello non ha riconosciuto («tenda da sole», «skylight»).

13. Il quinto ciclo: nove lingue

I cicli 1–4 coprivano italiano, inglese e spagnolo. Il quinto ciclo aggiunge francese, tedesco, olandese, portoghese, polacco e svedese, scelte per la diffusione di Home Assistant in Europa, e ha un secondo obiettivo: spostare nel modello regole che finora dipendevano da elenchi di parole in italiano, inglese e spagnolo. Per questo introduce la testa di etichette delle parole (per dividere i comandi multipli e trovare i valori in qualunque lingua) e i tipi CONFIRM e HISTORY. Una regola generica sostituisce, per le lingue nuove, quella basata sui nomi generici: se la frase non contiene nessuna parola che distingua il dispositivo scelto dagli altri dello stesso tipo e il modello dà a un'alternativa almeno il 15% della probabilità del dispositivo scelto, il sistema chiede.

Punto di partenza. Abbiamo tradotto la casa di prova nelle sei lingue nuove (39 frasi per lingua) e valutato il modello del ciclo 4, che non le ha mai viste (barre grigie della Figura 21). Le prestazioni crollano, dal 94,5% dell'italiano a percentuali tra il 23% e il 51%, e soprattutto compaiono molte esecuzioni sbagliate: 32 nelle sei lingue nuove, di cui 11 in polacco.

Dati. Il generatore produce 350 case nuove per ciascuna lingua; il LLM genera circa 20.900 parafrasi verificate e circa 5.100 frasi fuori tema, complesse o senza azione nelle lingue nuove. Per insegnare al modello gli errori veri del riconoscimento vocale, fino a 1500 frasi per lingua (circa 11.300 in tutto) vengono lette da una voce sintetica (Piper) e trascritte da Whisper *small*: la trascrizione prende il posto della frase, con le stesse etichette. Whisper a volte non trascrive ma inventa una frase diversa (per esempio «bitte ändern die Lautstärke» → «wenn es nicht so ist dann ist es so»). Le trascrizioni con

una somiglianza di caratteri inferiore a 0,7 rispetto all'originale vengono scartate: il 31% in tedesco, il 20% in portoghese e tra l'1,6% e il 3,4% nelle altre lingue, lasciandone 9.556. La Figura 20 mostra la composizione finale.

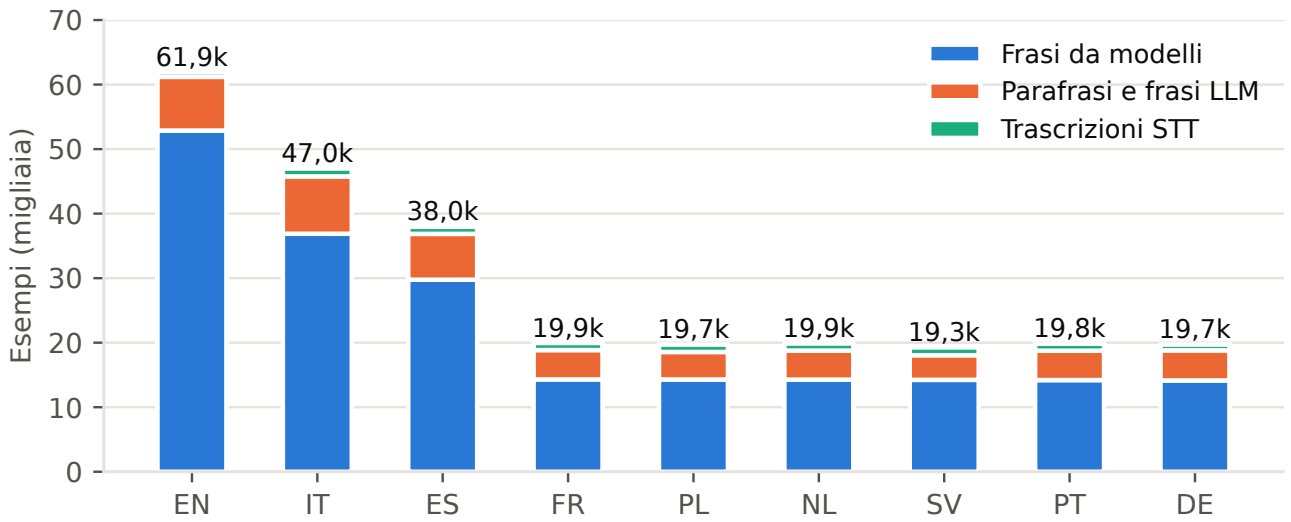


Figura 20. Esempi di addestramento del ciclo 5 per lingua e per origine; le trascrizioni della voce sintetica (circa 1000 per lingua) sono lo strato più sottile, in cima. Inglese, italiano e spagnolo hanno più esempi perché includono le case e le parafrasi dei cicli precedenti.

13.1 Risultati del modello e perché solo 6 strati

L'insegnante riaddestrato in 9 lingue mette al primo posto la risposta corretta nel 98,5% dei casi di validazione e riconosce il tipo di richiesta nel 99,5%. Gli studenti a 6 e 4 strati sono stati addestrati sugli stessi dati (Tabella 13). La 4 strati è più veloce di un terzo, ma sulle case di prova tradotte esegue per sbaglio quasi il doppio dei comandi (11 contro 6) e perde su tutti gli insiemi di prova. Poiché un'esecuzione sbagliata cambia davvero qualcosa in casa, dal ciclo 5 teniamo solo la variante a 6 strati: la 4 strati non viene più addestrata né distribuita, e il pacchetto per Home Assistant contiene un solo modello.

Tabella 13. Le due varianti del ciclo 5, con il software di runtime di allora. Case di prova: 362 frasi nelle 7 versioni tradotte. Latenze sul PC (CPU, 4 thread).

	e5s-6L	e5s-4L
Gold: accuratezza / precisione delle esecuzioni	96,9% / 100%	96,0% / 99,3%
Test sintetico (15.000 frasi): accuratezza / precisione	92,9% / 97,5%	92,5% / 96,8%
Case di prova in 7 lingue: corrette / esecuzioni sbagliate	89,2% / 6	88,1% / 11
Latenza p50 / p95	41,4 / 66,6 ms	29,3 / 47,8 ms
File INT8	24,3 MB	20,6 MB

13.2 Correzioni del software dopo il ciclo 5

Gli errori rimasti sulle case di prova avevano quasi tutti cause nel software intorno al modello, non nel modello, e sono stati corretti senza riaddestrare (versioni 0.6.0–0.6.3 dell'add-on):

- **Colori e valori.** Le case salvate prima del ciclo 5 conoscevano i nomi dei colori solo in tre lingue; ora le parole delle altre lingue vengono aggiunte al caricamento. La lettera polacca «ł» (e «ß», «ø») veniva spezzata in due parole e «do połowy» (a metà) non era riconosciuto.
- **Comandi doppi.** La testa di etichette segna raramente la congiunzione come separatore; ora la frase si divide sulle congiunzioni delle nove lingue, e le parole ambigue («i» articolo italiano e

congiunzione polacca, «en» preposizione spagnola e congiunzione olandese) valgono come separatore solo nella lingua della frase e solo se segue un verbo.

- **Nomi e stanze.** Se la frase dice il nome intero di un dispositivo, o è una frase generica («accendi la luce») detta in una stanza con un solo dispositivo adatto, la scelta è certa anche quando la probabilità è appena sotto la soglia; se la frase contiene una parola che distingue un altro dispositivo e nessuna di quello scelto, il sistema chiede invece di eseguire; se la frase è esattamente il nome di una scena, la scena viene attivata.
- **Condizioni.** Le regole che passano all'LLM le richieste condizionali («se piove...», «wenn es regnet...») esistono ora in tutte le nove lingue.

La lingua come parametro. L'integrazione con Assist conosce la lingua della frase. Abbiamo verificato se convenisse usarla anche per calibrare il modello lingua per lingua: la temperatura stimata sulle sole frasi italiane di validazione è 1,60, identica a quella globale, e soglie di esecuzione più basse per alcune lingue recuperano 8 frasi su 1500 sintetiche al prezzo di 3 esecuzioni sbagliate in più. Entrambe le idee sono state scartate; la lingua viene usata solo per leggere le congiunzioni ambigue.

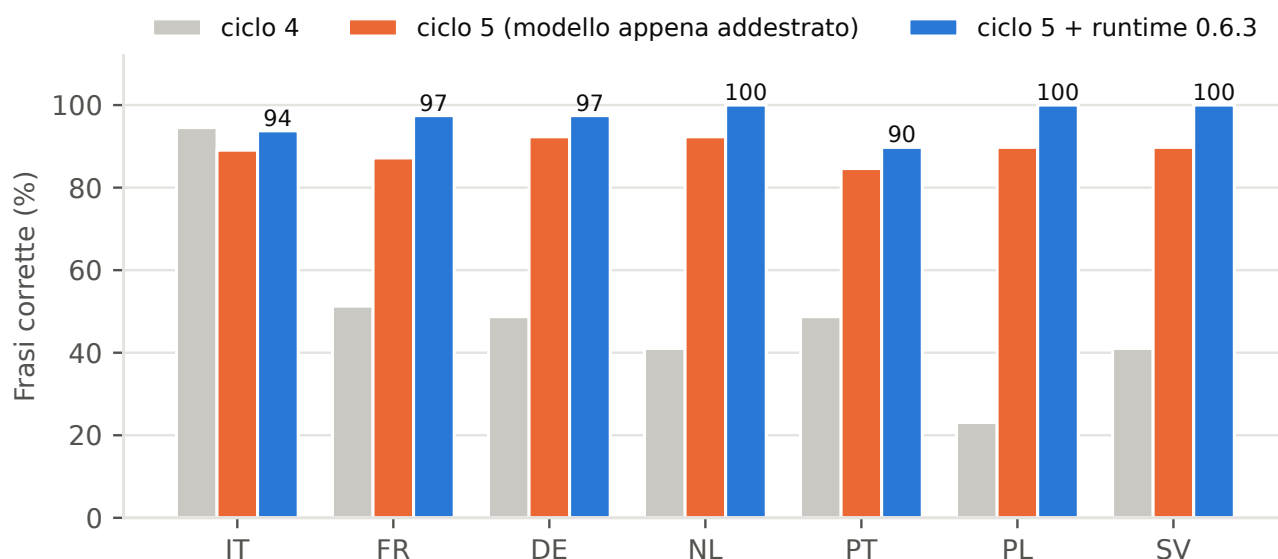


Figura 21. Frasi corrette sulle case di prova in ogni lingua: modello del ciclo 4, modello del ciclo 5 con il software di allora e con il software 0.6.3. IT: 128 frasi; altre lingue: 39 ciascuna. Esecuzioni sbagliate in totale: 33, 6 e 2. Le correzioni sono state progettate guardando gli errori di queste stesse frasi, quindi l'ultima serie è ottimistica; la verifica indipendente è il gold (222/227, nessuna esecuzione sbagliata).

13.3 Prova in tutte le lingue su Raspberry Pi e Mac

Il modello del ciclo 5 con il software 0.6.3 è stato provato sulle case di prova in tutte le nove lingue, sia sul Raspberry Pi 5 dentro Home Assistant sia su un Mac con processore Apple, con le stesse 404 frasi (Tabella 14). I due risultati coincidono quasi del tutto: le piccole differenze vengono dall'aritmetica INT8, diversa tra i due processori, che sposta di poco i punteggi vicini alle soglie. Le due esecuzioni sbagliate rimaste sono errori del modello a basso rischio: «chiudi il lucernario della mansarda» spegne la luce della mansarda e «liga a máquina de lavar loiça» (lavastoviglie, assente) accende la lavatrice. Tutti gli altri errori sono domande o rifiuti.

Tabella 14. Frasi corrette per lingua, modello del ciclo 5 con il software 0.6.3. La casa italiana contiene anche le frasi in inglese e spagnolo. Tempo mediano: sul Mac la sola decisione, sul Pi anche il viaggio in rete dal Mac.

Lingua	Frasi	Mac (Apple, arm64)	Raspberry Pi 5
Italiano	130	127	125
Inglese	21	17	17
Spagnolo	19	15	16
Francese	39	38	38
Tedesco	39	38	38
Olandese	39	39	39
Portoghese	39	35	35
Polacco	39	39	38
Svedese	39	39	39
Totale	404	387 (95,8%)	385 (95,3%)
Esecuzioni sbagliate		2	2
Tempo mediano		36 ms	174 ms

Sulla casa di prova di Home Assistant (170 frasi in italiano, inglese e spagnolo) il ciclo 5 con il software 0.6.0 ha ottenuto 150/170 (88,2%), contro 160/170 del ciclo 4, con 20 scenari su 20 eseguiti correttamente sui dispositivi virtuali e 5 prove su 5 attraverso Assist; con le correzioni 0.6.1–0.6.3 le stesse frasi salgono a 158/170 sul Pi. Il prezzo delle sei lingue nuove è quindi di circa un punto in italiano, inglese e spagnolo.

14. Il sesto ciclo: meteo e musica per nome

Il sesto ciclo aggiunge tre funzioni che si chiedono ogni giorno a un assistente vocale. La prima è il **meteo**: condizioni attuali e previsioni per un giorno, lette dall'entità meteo di Home Assistant e raccontate nella lingua della domanda. La seconda è la **musica per nome**: frasi come «metti radio deejay in cucina», «riproduci bohemian rhapsody» o «metti la playlist relax» sui lettori di Music Assistant, che cercano per nome brani, artisti, playlist e radio su Spotify, Radio Browser, TuneIn o file locali. Il modello impara a marcare come VALORE le parole del contenuto («radio deejay»), che vengono passate così come sono alla ricerca: EdgeDecision sceglie solo il lettore. La terza riguarda **timer e sveglie**, che il modello impara a riconoscere e a passare all'agente di Home Assistant. Il ciclo aggiunge anche dati mirati sui casi rimasti deboli nel ciclo 5. Per misurare le funzioni nuove abbiamo scritto un insieme di prova «r6» di 105 frasi nelle nove lingue: 15 in italiano, 14 in ciascuna delle sei lingue nuove, 4 in inglese e 2 in spagnolo.

Cancelli di accettazione. Dal sesto ciclo un modello nuovo sostituisce quello in uso solo se supera dei controlli automatici, sugli stessi insiemi di prova e con lo stesso software:

- **Gold:** nessuna esecuzione sbagliata in più e al massimo un punto di accuratezza in meno.
- **Case di prova:** al massimo una frase corretta in meno per ogni lingua e non più esecuzioni sbagliate in totale.
- **r6:** almeno l'85% di frasi corrette e nessuna esecuzione sbagliata.
- **Latenza:** latenza mediana al massimo il 15% più alta.

Tabella 15. Ciclo 5 e ciclo 6 sugli stessi insiemi di prova, misurati sul PC. Case di prova: 404 frasi in 9 lingue; r6: 105 frasi in 9 lingue.

	Ciclo 5 + 0.6.3	Ciclo 6 + 0.6.3	Ciclo 6 + 0.7.0	Ciclo 6 + 0.7.1
Gold (227): corrette	222	218	221	220
Case di prova: corrette	386	387	393	393
Case di prova: esecuzioni sbagliate	2	2	1	1
r6: corrette	11	80	103	103
r6: esecuzioni sbagliate	9	0	0	0
Cancelli superati	–	no (3 falliti)	sì	sì

Risultato del modello. Il modello del ciclo 5 sbagliava quasi tutte le frasi r6 e ne eseguiva male 9. Il modello del ciclo 6, così come esce dall'addestramento, non esegue mai l'azione sbagliata e ne risolve 80 su 105. Non passa però tre cancelli: il gold scende a 218, il tedesco perde due frasi e r6 resta sotto l'85% (Tabella 15). La latenza mediana non cambia (37 → 38 ms sul PC).

Correzioni del software (0.7.0). Gli errori su r6 avevano quasi tutti la stessa causa. Il modello riconosceva il contenuto, ma la testa del tipo di richiesta classificava come COMPLESSE frasi come «metti un po' di jazz» o «lance ma playlist détente», e una regola le passava all'LLM. Altre volte una radio dal nome sconosciuto («rmf fm», «mix megapol») veniva rifiutata come non supportata. Senza riaddestrare abbiamo cambiato il software intorno al modello:

- **Prima il contenuto.** Se la testa delle etichette trova parole di contenuto e la casa ha un lettore che cerca per nome, la frase diventa una riproduzione di quel contenuto, a meno che il modello non la giudichi chiaramente fuori dominio.
- **Musica in una stanza.** «Metti musica in cucina» su un lettore di Music Assistant fa partire la musica invece di accendere soltanto il lettore.
- **Meteo, timer e sveglie** sono esclusi dalle regole che passano all'LLM le richieste complesse.
- **Stanza diversa.** Se il contenuto è chiesto per una stanza senza lettori, il sistema chiede invece di usare il lettore di un'altra stanza.

Con il software 0.7.0 tutti i cancelli sono superati: 221 frasi su 227 sul gold, 393 su 404 sulle case di prova con una sola esecuzione sbagliata, 103 su 105 su r6. Sul Raspberry Pi 5 dentro Home Assistant le stesse prove danno 394 su 404 e 103 su 105, con una latenza mediana di 147 ms; sul Mac 394 su 404 in 36 ms. Come nel ciclo 5, le correzioni sono state progettate guardando gli errori di queste frasi, quindi i numeri delle case di prova e di r6 sono ottimistici; la verifica indipendente resta il gold. Gli errori rimasti sono quasi tutti domande o rifiuti: «open the skylight» e «cierra la claraboya» non trovano ancora il lucernario, che ha solo un nome italiano, e «apri il cancello» chiede invece di rifiutare. L'unica esecuzione sbagliata è ancora «liga a máquina de lavar loiça»: la lavastoviglie non c'è e il sistema accende la lavatrice.

14.1 Domande sullo stato (0.7.1)

Il modello sa leggere lo stato di un singolo dispositivo («la finestra della cucina è aperta?»). La versione 0.7.1 estende le domande senza riaddestrare, con regole nel software dopo la decisione del modello:

- **Stanza.** «La luce del salone è accesa?» in una stanza con più luci risponde con lo stato di tutte.
- **Tutta la casa.** «Ci sono luci accese?», «quali finestre sono aperte?», «quante luci sono accese?» producono l'elenco o il conteggio su luci, prese, tapparelle, porte e finestre, serrature, TV e casse, ventilatori.

- **Solo la stanza.** «Accendi in cucina», senza dire cosa, accende le luci della stanza.

Le risposte sono composte dal software nelle nove lingue, con genere e numero corretti («la finestra è aperta», «le finestre sono aperte»). Sugli stessi insiemi la 0.7.1 ottiene 220/227 sul gold, 393/404 sulle case di prova e 103/105 su r6, senza peggiorare ciò che già funzionava.

15. Il settimo ciclo: tutta la casa, seconde case, timer

Il settimo ciclo allarga il dominio a tutti i tipi di abitazione: appartamenti, case su più piani, seconde case al mare o in montagna, hotel. Ha aggiunto:

- **Tutta la casa e i piani:** «spegni tutto», «spegni le luci del primo piano», «cosa è rimasto acceso?», «chi è a casa?». «Spegni tutto» agisce solo su luci, lettori, ventilatori e prese non protetti: non tocca mai serrature, allarmi, valvole, frigoriferi o caldaie.
- **Dispositivi delle seconde case:** valvole dell'acqua e del gas, scaldabagno, preset del clima (eco, comfort, assenza), umidificatori, sensori di perdita, fumo, movimento e consumo, pulsanti del cancello, lamelle delle veneziane, robot tosaerba.
- **TV e automazioni:** scelta della sorgente («metti Netflix sulla tv»), riproduzione casuale, script e automazioni, presenza delle persone.
- **Timer, sveglie, annunci e lista della spesa, gestiti direttamente.** Invece di passarli a un altro agente, EdgeDecision li riconosce e legge durate e orari nelle nove lingue, comprese forme come «un quarto alle otto», «halb acht» o «wpół do ósmej». Poi chiama gli intenti nativi di Home Assistant sul satellite che ha sentito la frase, così il timer suona sul Voice PE giusto.
- **Forme di cortesia e richieste da hotel.** «Potrebbe accendere la luce?» viene eseguita come un comando; «posso avere degli asciugamani puliti?» passa all'altro agente invece di ricevere un «non posso». Il ciclo aggiunge anche varianti regionali e corregge i casi rimasti deboli.

Due entità virtuali rappresentano le funzioni che non appartengono a un dispositivo: «Assistente vocale» (timer, sveglie, annunci) e «Tutta la casa» (spegni tutto, stato della casa); una terza, «altro agente», raccoglie ciò che la casa non sa fare da sola (il meteo senza un'entità meteo, la musica senza Music Assistant). Così restano capacità come le altre, scelte dallo stesso modello.

Dal software al modello. Il ciclo 7 insegna al modello le scelte che nelle versioni 0.7.x facevano le regole: stanza o casa intera nelle domande sullo stato, stato della TV, comandi con la sola stanza, riconoscimento del contenuto. Restano nel software le parti deterministiche, quelle in cui una regola è più affidabile di un modello: il testo delle risposte, la lettura di numeri, durate e orari, le esclusioni di sicurezza e le chiamate agli intenti.

Dati e prova. Le frasi nuove vengono da modelli scritti a mano in ciascuna delle nove lingue e non dal LLM; le parafrasi dei cicli precedenti vengono riutilizzate, tranne quelle la cui etichetta cambia. Il nuovo insieme di prova «r7» ha 215 frasi nelle nove lingue. Il modello del ciclo 6 ne risolve 82 ed esegue male 16: le funzioni nuove non si possono aggiungere solo con il software. Ai cancelli del ciclo 6 si aggiungono quelli su r7: almeno l'85% di frasi corrette, almeno il 75% in ogni lingua e nessuna esecuzione sbagliata.

15.1 Tre addestramenti: 7, 7b e 7c

Ciclo 7. Il primo addestramento ha imparato buona parte delle funzioni nuove (192 frasi r7 su 215), ma ha fallito dieci cancelli: il gold è sceso da 220 a 206 frasi, cinque lingue delle case di prova hanno perso più di una frase, r6 è sceso da 103 a 84 e sono comparse esecuzioni sbagliate (7 su r7). L'analisi ha trovato una causa nei dati: le case di addestramento non avevano le entità virtuali che il software

aggiunge in Home Assistant, quindi a runtime il modello vedeva candidati che non aveva mai incontrato, e una richiesta di meteo o di musica in una casa senza meteo o senza Music Assistant era etichettata come rifiuto.

Ciclo 7b. Il secondo addestramento (14 ore sulla RTX 3090) dà a ogni casa sintetica le stesse entità virtuali di Home Assistant e tratta le richieste che la casa non può soddisfare da sola come richieste da passare all'altro agente. Su r7 risolve 207 frasi su 215 senza esecuzioni sbagliate, ma sul gold resta sotto il ciclo 6 (212 frasi contro 218 con lo stesso software, e un'esecuzione sbagliata) e chiede «quale?» più spesso sui comandi di base. La versione 0.8.0 dell'add-on lo ha quindi distribuito come modello facoltativo, accanto a quello del ciclo 6.

Ciclo 7c. Il terzo addestramento (14,4 ore) parte dagli errori del 7b e dalle prime sessioni reali sul Raspberry Pi dell'autore, con EdgeSTT. Toglie due conflitti di etichetta: le richieste di musica come «metti un po' di jazz» comparivano anche nell'elenco delle richieste complesse, e in spagnolo «vale» era sia «sì» sia «annulla». Aggiunge poi le frasi sentite davvero e le loro varianti: «a che ora è la sveglia?» in molte forme, anche come le scrive il riconoscimento vocale («a che orai la sveglia»); orari detti cifra per cifra («alle tredici zero cinque»); lo storico chiesto senza verbo («ultimi comandi eseguiti»); frasi lunghe sentite da una TV o da una radio, che devono restare fuori dominio; scene chiamate in un'altra lingua; lettori con un nome musicale («Musica»); più esempi per le case in spagnolo e in inglese, le più deboli nelle prove del 7b. Un nuovo insieme di prova «r7c» di 48 frasi misura questi casi.

Soglie. La calibrazione del 7c ha scelto soglie di ambiguità e di tipo di richiesta più alte di quelle del 7b (rapporto di ambiguità 0,45 invece di 0,30; soglie per le richieste complesse e senza azione 0,5 e 0,8 invece di 0,6 e 0,6). A parità di modello e di software, con le soglie del 7b il gold sale da 211 a 214 frasi, mentre gli altri insiemi restano uguali o perdono una frase. La versione 1.0 usa il modello 7c con le soglie del 7b e la temperatura stimata per il 7c (1,45).

Tabella 16. Frasi corrette sugli insiemi di prova, misurate sul PC. Ciclo 6 con il software in uso il 2 ottobre (0.7.x); cicli 7b e 7c con il software 0.8.4, quello della versione 1.0. Il 7c usa le soglie del 7b.

Insieme	Ciclo 6	Ciclo 7b	Ciclo 7c (1.0)
Gold (227 frasi, IT/EN/ES)	220	212	214
Case di prova (404 frasi, 9 lingue)	392	387	385
r6 (105 frasi)	103	95	99
r7 (215 frasi)	82	207	206
r7c (48 frasi)	–	35	46
Esecuzioni sbagliate, tutti gli insiemi	17	2	0
Latenza mediana sul PC	38 ms	40 ms	39 ms

Il 7c con il software 0.8.4 supera tutti i cancelli tranne uno: lo spagnolo delle case di prova scende da 15 a 13 frasi su 19 (le altre lingue: italiano 125/130, inglese 17/21, francese, tedesco e olandese 39/39, portoghese 37/39, polacco e svedese 38/39). Gli errori spagnoli sono tutti prudenti: domande o rifiuti, quasi tutti su dispositivi che hanno solo un nome italiano («claraboya» per il lucernario, «buhardilla» per la mansarda, «cafetera» per la macchina dell'espresso). Abbiamo accettato questa eccezione perché il 7c elimina le esecuzioni sbagliate, risolve 11 frasi r7c in più e migliora il gold.

16. Il software 0.8: dalle prove in casa

Tra il 2 e il 4 ottobre 2026 l'autore ha usato il sistema nella propria casa, con EdgeSTT e un satellite Voice PE, in tutte le nove lingue, salvando le decisioni (opzione `save_history`). Gli errori di queste sessioni hanno prodotto le versioni 0.8.1–0.8.4 dell'add-on, solo software, valide per qualunque modello:

- **Orologio.** «Che ore sono?» e «che giorno è?» ricevono la risposta dall'orologio del Raspberry, nelle nove lingue, anche nelle forme storpiate dal riconoscimento vocale («che ore so no», «wie spät ist das»). Valgono solo le frasi brevi, così «a che ora è il tramonto?» non diventa una domanda sull'ora.
- **Il satellite come contesto.** L'integrazione manda all'add-on il dispositivo del satellite che ha sentito la frase. «Mettili radio deejay» suona sul suo altoparlante senza chiedere dove; «abbassa il volume» regola quell'altoparlante e non una TV accesa in un'altra stanza; «ferma la musica» non ferma mai una TV. Il lettore ESPHome del Voice PE e il suo gemello di Music Assistant vengono riconosciuti come lo stesso altoparlante.
- **Protezioni.** Le ultime esecuzioni sbagliate sugli insiemi di prova sono diventate regole generali: una parola di sblocco, in nove lingue, impedisce di chiudere a chiave («doe de voordeur van het slot»); un elettrodomestico nominato e assente non ne accende un altro («liga a máquina de lavar loiça»); un oggetto nominato che non compare nel nome del dispositivo scelto porta a una domanda («chiudi il lucernario» non spegne la luce della mansarda).
- **Dialogo.** Dopo una domanda di chiarimento, un comando nuovo e chiaro viene eseguito come tale invece di essere preso per una risposta; una sola parola del nome basta per scegliere tra le opzioni; una domanda su un solo dispositivo nomina l'azione («Vuoi spegnere le luci in Camera?»), così un «sì» non può fare il contrario di quello che si voleva.
- **Errori di Home Assistant.** Se un servizio fallisce il sistema non dice più «Fatto.» ma spiega cosa non è andato («Non ho trovato Radio DJ.»).
- **Errori del riconoscimento in streaming.** Alcuni verbi storpiati da EdgeSTT («spende», «stenga», «ascenda») vengono corretti prima del modello, come già avveniva per gli errori tipici di Whisper.
- **Prudenza per impostazione predefinita.** «Spegni tutto» resta disattivato finché l'utente non lo abilita, perché una frase capita male non deve spegnere la casa, e lo storico delle frasi viene salvato solo su richiesta.

Come per le versioni precedenti, alcune di queste regole sono state verificate sulle stesse frasi che le hanno suggerite; il loro effetto è compreso nei numeri della Tabella 16.

17. Confronto con l'agente predefinito di Home Assistant

Per capire quanto valga l'approccio rispetto a ciò che un utente di Home Assistant ha già, abbiamo misurato l'agente di conversazione predefinito sulle stesse frasi e con lo stesso criterio di correttezza.

Metodo. Le frasi sono le 846 dei tre insiemi scritti a mano gold, case di prova e r7, in nove lingue e mai usate in addestramento; circa un quarto chiede qualcosa che *non* va eseguito (fuori dominio, non supportato, ambiguo, ad alto rischio). L'agente di Home Assistant è riprodotto con i suoi componenti: i modelli di frase ufficiali (`home-assistant-intents` 2026.9.30) riconosciuti da `hassil` 3.12.1 e riempiti con i nomi dei dispositivi, delle stanze e dei piani delle stesse case. Un intento riconosciuto viene risolto come fa Home Assistant: un nome di dispositivo indica quel dispositivo; una stanza e un tipo indicano i dispositivi di quel tipo nella stanza; un tipo da solo indica la stanza del microfono, altrimenti tutta la casa. I comandi ritardati («spegni la luce tra dieci minuti») contano come corretti, perché Home Assistant li esegue con un timer. Un comando è corretto se viene eseguita l'azione giusta sul dispositivo

giusto; una frase da non eseguire è corretta se non viene eseguito nulla. Non abbiamo riprodotto la corrispondenza approssimata dei nomi delle versioni recenti di Home Assistant, le frasi personalizzate né gli agenti LLM.

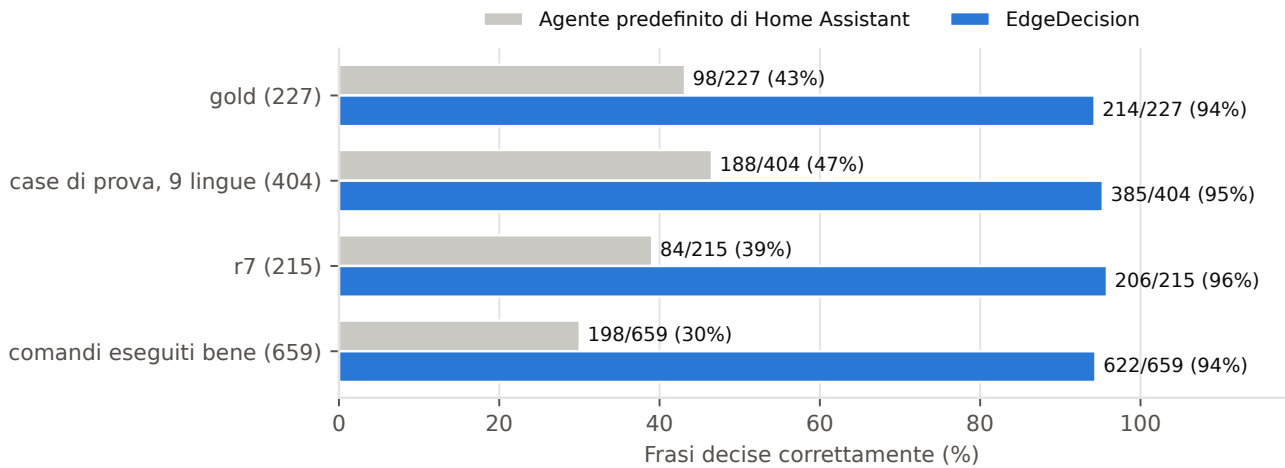


Figura 22. Frasi decise correttamente dall'agente predefinito di Home Assistant e da EdgeDecision (modello 7c, software 1.0) sugli stessi insiemi. L'ultima riga conta solo le frasi che chiedono un'azione da eseguire.

Tabella 17. Agente predefinito di Home Assistant ed EdgeDecision sulle stesse 846 frasi.

	Home Assistant	EdgeDecision
Gold (227)	98 (43%)	214 (94%)
Case di prova, 9 lingue (404)	188 (47%)	385 (95%)
r7 (215)	84 (39%)	206 (96%)
Totale (846)	370 (44%)	805 (95%)
Comandi eseguiti bene (659)	198 (30%)	622 (94%)
Esecuzioni sbagliate	0	0
Azioni ad alto rischio eseguite senza conferma	15	0
Solo le frasi nella stessa lingua dei nomi della casa		
Gold	58 / 106	102 / 106
Case di prova	178 / 364	355 / 364
r7	73 / 196	187 / 196
Tempo di decisione, CPU del PC (2 thread)	2–12 ms	circa 120 ms

L'agente di Home Assistant non sbaglia mai dispositivo: quando la frase non combacia con un modello non fa nulla. Il suo limite è la copertura. Basta un piccolo cambiamento di formulazione perché nessun modello combaci: «spegni l'abat-jour *di* Paolo» per un dispositivo chiamato «Abat-jour Paolo», «abbassa *un po'* la luce», «prossima canzone», «è buio qui», «could you turn the living room lamp on please». Le frasi in una lingua diversa da quella dei nomi lo penalizzano, perché nomi e stanze devono comparire come sono scritti; ma anche contando solo le frasi nella stessa lingua dei nomi il divario resta quasi identico. La differenza sulla sicurezza è di natura diversa: un modello di frase esegue «apri il cancello» o «sblocca la porta» appena la riconosce, mentre EdgeDecision chiede sempre conferma per le azioni ad alto rischio. Il vantaggio dell'agente di Home Assistant è la velocità: da 2 a 12 ms quando un modello combacia, contro circa 120 ms di EdgeDecision sulla stessa CPU. Le frasi di prova sono state scritte dagli autori di EdgeDecision e non sono pubblicate; il metodo e i conteggi lo sono, così che il confronto possa essere ripetuto su qualunque insieme di frasi.

18. La catena completa sul Raspberry Pi 5

L'ultima misura riguarda il sistema come lo usa una persona: un Raspberry Pi 5 (16 GB) con Home Assistant OS, un satellite Voice PE, EdgeSTT 0.2.2 (blocchi da 560 ms, 4 thread) ed EdgeDecision 0.8.3 (modello 7c, 4 thread) attivi insieme. Il 4 ottobre 2026 l'autore ha pronunciato 50 comandi in italiano; i tempi vengono dai registri dei due add-on (Tabella 18, Figura 23).

Tabella 18. La catena sul Raspberry Pi 5: una sessione di 50 comandi in italiano, 4 ottobre 2026.

Passo	Mediana	90% dei comandi sotto
EdgeSTT: dalla fine del parlato al testo (finalizzazione)	276 ms	519 ms
EdgeDecision: dal testo alla decisione	203 ms	510 ms
Dalla fine del parlato alla decisione (somma delle mediane)	$\approx 0,48$ s	–
Memoria: EdgeSTT / EdgeDecision		1,40 GB / 0,32 GB
CPU media tra due letture a 12 minuti di distanza: EdgeSTT / EdgeDecision		0,22 / 0,06 core su 4

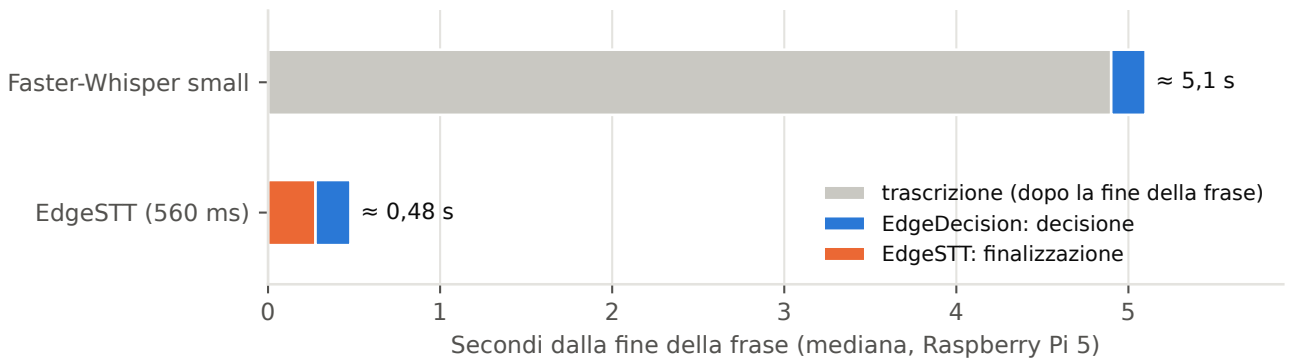


Figura 23. Tempo dalla fine della frase alla decisione sul Raspberry Pi 5 (mediane). In basso la catena misurata con EdgeSTT ed EdgeDecision; in alto, per confronto, la sola trascrizione di Faster-Whisper small (circa 4,8–5,0 s su frasi brevi, misurata in una prova precedente) più la stessa decisione. Le due misure non vengono dalla stessa sessione.

La finalizzazione di EdgeSTT ha due valori tipici, circa 250 e circa 510 ms, secondo che alla fine del parlato restino uno o due blocchi da elaborare: il resto della frase era già stato trascritto mentre l'utente parlava. Il tempo di decisione di EdgeDecision (mediana 203 ms) è più alto dei circa 150 ms misurati senza EdgeSTT, perché i due add-on si dividono i quattro core; in una sessione precedente in nove lingue, con il modello 7b, la mediana era 234 ms (p95 485 ms) su 213 frasi. A questi tempi vanno aggiunti il rilevamento della fine del parlato sul satellite e la sintesi vocale, che non dipendono dai due add-on e non sono stati misurati.

Cosa è successo nella sessione. 5 registrazioni su 50 sono uscite vuote dal riconoscimento vocale e non hanno prodotto azioni; diverse contenevano errori di trascrizione («Spende la luce» per «spegni la luce», «Impas on tainer di due minuti» per «imposta un timer di due minuti»). Il registro di EdgeDecision contiene 49 decisioni: 32 esecuzioni, 13 domande all'utente, 3 risposte senza azione («Grazie», «what did I just ask?») e 1 richiesta passata all'altro agente. Non abbiamo annotato quante di queste decisioni fossero quelle volute: la sessione misura tempi e risorse, non l'accuratezza.

19. Licenze

EdgeDecision ed EdgeSTT sono pubblicati come software **a sorgente disponibile per uso non commerciale**, non open source nel senso OSI. Il codice dei due add-on e dell'integrazione è concesso con la licenza PolyForm Noncommercial 1.0.0; il modello di EdgeDecision e la documentazione, compreso questo documento, con la licenza Creative Commons Attribuzione – Non commerciale 4.0 (CC BY-NC 4.0). Sono ammessi l'uso personale, hobbistico, di ricerca e didattico; l'uso commerciale no. I dati, gli strumenti di addestramento e le frasi di prova non sono pubblicati.

Il modello di EdgeDecision deriva da `multilingual-e5-small`, rilasciato da Microsoft con licenza MIT, che permette modifica e redistribuzione a condizione di includere l'avviso di copyright e il testo della licenza: il repository li contiene nel file `THIRD_PARTY_LICENSES.md`. Il modello di EdgeSTT non fa parte del repository: Nemotron 3.5 ASR Streaming 0.6B è concesso da NVIDIA con la licenza OpenMDW-1.1, che ne permette anche l'uso commerciale, e l'add-on lo scarica al primo avvio dalla release di sherpa-onnx. La licenza non commerciale copre quindi il codice di EdgeSTT, non i pesi del modello vocale; il repository include il testo di OpenMDW-1.1 e un file `NOTICE` con le attribuzioni.

Gli altri componenti della Tabella 19 servono solo all'addestramento e non vengono distribuiti. L'insegnante è usato solo per produrre i punteggi su cui lo studente si addestra; i suoi pesi non entrano nel modello distribuito. Le 18 voci Piper usate per leggere le frasi (due per lingua) hanno licenze diverse, che dipendono dal materiale con cui ogni voce è stata registrata: la maggior parte è CC0 o CC BY, una è AGPL, due non dichiarano una licenza, e molte derivano dalla voce inglese *lessac*, i cui dati sono concessi solo per ricerca. L'audio è stato generato solo per ottenere trascrizioni ed è stato poi scartato: nessun audio e nessuna voce fanno parte del progetto, il modello legge testo e produce decisioni, e l'uso commerciale è escluso dalla licenza. L'elenco completo, voce per voce, è in `THIRD_PARTY_LICENSES.md`.

Tabella 19. Componenti di terze parti (verificato tra il 29 settembre e il 4 ottobre 2026; non costituisce parere legale).

Componente	Ruolo	Licenza	Distribuito
<code>multilingual-e5-small</code>	base del modello e5s-6L	MIT	sì (derivato)
Nemotron 3.5 ASR Streaming 0.6B	modello di EdgeSTT	OpenMDW-1.1	no: scaricato al primo avvio
sherpa-onnx, wyoming	runtime di EdgeSTT	Apache 2.0, MIT	installati dal pacchetto
onnxruntime, tokenizers, numpy, num2words	runtime di EdgeDecision	MIT, Apache 2.0, BSD, LGPL	installati dal pacchetto
<code>bge-reranker-v2-m3</code>	insegnante per la distillazione	Apache 2.0	no
Qwen2.5-14B-Instruct	parafrasi e frasi generate	Apache 2.0	no
Whisper small	trascrizione delle frasi lette	MIT	no
Piper e 18 voci	lettura delle frasi	MIT; voci: CC0, CC BY, AGPL, solo ricerca (<i>lessac</i>) o non dichiarata	no

20. Discussione e limiti

- **Dati sintetici e prove degli autori.** Tutto l'addestramento usa case e frasi generate. Gli insiemi di prova (846 frasi più r6 e r7c) sono scritti dagli stessi sviluppatori, che vi hanno anche guardato gli errori per correggere il software; le regole delle versioni 0.6–0.8 sono quindi in parte misurate sulle frasi che le hanno suggerite, e i numeri vanno letti come ottimistici. Una valutazione con utenti diversi dall'autore e con registrazioni annotate manca ancora.
- **Errori di trascrizione.** EdgeDecision è stato addestrato a reggere errori di riconoscimento simulati con Whisper, ma nell'uso reale il testo arriva da Nemotron, che sbaglia in modo diverso (parole storpiate come «spende» per «spegni», testi vuoti sulle risposte brevi). Addestrare EdgeDecision sugli errori di EdgeSTT, e EdgeSTT sui comandi domestici, sono i primi miglioramenti previsti.
- **Lingue.** Lo spagnolo è la lingua più debole di EdgeDecision sulle case di prova (13 frasi su 19) e l'inglese segue; nel modello vocale, polacco e svedese hanno un'accuratezza dichiarata molto inferiore alle altre lingue. Le frasi in una lingua su dispositivi che hanno solo un nome in un'altra («skylight» per «Velux», «claraboya» per «lucernario») restano difficili.
- **Calibrazione.** Le soglie della versione 1.0 sono quelle del ciclo 7b, scelte confrontando due configurazioni sugli insiemi di prova e non solo sul validation; sul validation sintetico la precisione delle esecuzioni resta sotto l'obiettivo del 99%.
- **Generalità.** EdgeDecision sa solo controllare una casa. Le richieste fuori dominio e quelle complesse («accendi il riscaldamento quando arrivo a casa») vengono riconosciute e passate all'agente di riserva, non risolte.
- **Confronti.** Il confronto con Home Assistant riproduce l'agente predefinito senza la corrispondenza approssimata dei nomi, le frasi personalizzate e gli agenti LLM; un LLM locale sul Raspberry Pi e i modelli decisionali generali come Jev e CLM non sono stati misurati sugli stessi test.
- **Misure sul campo.** I tempi della catena vengono da una sola sessione di 50 comandi in italiano di un solo parlante; non abbiamo misurato il rilevamento della fine del parlato sul satellite, la sintesi vocale né l'effetto di più satelliti che parlano insieme (EdgeSTT decodifica una richiesta alla volta).

21. Conclusioni

EdgeDecision ed EdgeSTT mostrano che un assistente vocale domestico può essere veloce, locale e prudente insieme, su un Raspberry Pi 5. EdgeDecision affida la comprensione a un modello di 23,9 milioni di parametri che sceglie tra le capacità reali della casa invece di generare testo: è il «System 1» della casa e lascia a un altro agente le richieste che richiedono ragionamento. Riduzione del vocabolario, potatura degli strati, distillazione e quantizzazione portano a un file di 24,3 MB che capisce nove lingue; con il modello del settimo ciclo prende la decisione giusta nel 95% delle 846 frasi di prova, senza esecuzioni sbagliate e chiedendo sempre conferma per serrature, garage e allarmi, mentre l'agente predefinito di Home Assistant, sulle stesse frasi, si ferma al 44%. EdgeSTT sposta il riconoscimento vocale durante il parlato: dalla fine della frase alla decisione passano circa 0,48 s in mediana, contro i circa 5 s della sola trascrizione con Faster-Whisper. I prossimi passi sono l'addestramento dei due modelli sugli errori e sulle frasi dell'uso reale, una valutazione con più parlanti e registrazioni annotate, e il confronto con un LLM locale.